ICPC Notebook

template
commands.sh 1
compile.sh 1
gen.cpp 1
hash.sh 2
hash_values.txt 2
stress_test.sh 2
template.hpp 2
vim_settings.sh 2
data-structure
2DSegtree.hpp 3
ConvexHullTrick.hpp 3
FastSet.hpp 4
LiChao.hpp 4
LiChaoDyn.hpp 5
Mosalgo.hpp 5
RollbackUnionFind.hpp 5
UnionFind.hpp 5
cht.hpp 6
cht_light.hpp 6
cht_monotone.hpp 6
fenwick_tree.hpp 7
lazy_segtree.hpp 7
segtree.hpp 8
math
BinaryGCD.hpp 8
EulerPhi.hpp 8
ExtGCD.hpp 8
Matrix.hpp 9
floor_sum.hpp 9
lagrange.hpp 9
linear_congruence.hpp 9
math_template.hpp 10
primitive_root.hpp 10
xorConvolution.hpp 10
modint
BarrettReduction.hpp 10
combination.hpp 10
convolution.hpp 10
modint.hpp 11
extra
modint_fast.hpp 11
FPS
FFT.hpp 11
FFT_fast.hpp 11
graph
TreeHash.hpp 12
Trie.hpp 12
centroid_decomposition.hpp 12
dijkstra.hpp 13
edge.hpp 13
eulertrail.hpp 13
hld.hpp 13
lowlink.hpp 14
matrix_tree.hpp 14
scc.hpp 14
flow
bipartite-matching.hpp 15
maxflow.hpp 15
mincostflow.hpp 15
燃やす埋める.md 16
string
KMP.hpp 16
Manacher.hpp 16
lcp_array.hpp 16
palindrome.hpp 16
rolling_hash.hpp 16
suffix_array.hpp 17
z_algorithm.hpp 17
geometry
memo
Bi-Connected-Components.hpp 17
Primes.md 18
dynamic_segtree.hpp 18
geometry.hpp 19
input.md 22
int128.hpp 23
memory.md 23
memsize.cpp 24
random.hpp 24
range_edge_graph.hpp 24

template

commands.sh

```sh
# 本番で打つコマンド集。このファイル自体は打たない / 実行しない
# 環境: Ubuntu 24.04 / g++ 13 / /bin/sh は dash
#   dash に time は無い。/usr/bin/time も入っていない

# デバッグ（範囲外・UB を行番号つきで出す）
g++ -std=gnu++20 -g -O1 -fsanitize=address,undefined \
    -Wall -Wextra -Wshadow -o a a.cpp

# 提出前（ジャッジと同じ）。compileg++ a.cpp でも同じ
g++ -x c++ -g -O2 -std=gnu++20 -static -o a a.cpp

# 時間（ms）
s=$(date +%s%N); ./a < in; e=$(date +%s%N)
echo $(((e - s) / 1000000))

# 256MB に収まるか。0 以外なら超過
# ASan 版では使えないので -O2 版で試す
(ulimit -v 262144; ./a < in); echo $?

# 深い再帰。シェルに1回だけ
ulimit -s 65532

# 手打ちしたライブラリの照合。貼り付け -> Ctrl + D
# 出た6文字を hash_values.txt と見比べる
tr -d '[:space:]' | md5sum | cut -c-6

# メモリ見積り: 個数 x バイト数 / 10^6 = MB
#   char,bool 1 / short 2 / int,float 4
#   ll,double,pair<int,int>,ptr 8 / i128 16
#   set,map 40 / uset,umap 32
#   vector<vector<>> は1行あたり +30
#   reserve なしの push_back はピーク 2.5 倍
#   256MB = int 6400万 = ll 3200万
```

compile.sh

```sh
# 使い方:
#   sh compile.sh a.cpp     デバッグ（ASan + UBSan + 警告）
#   sh compile.sh a.cpp r   提出前（ジャッジと同じ -O2 のみ）
#   sh compile.sh a.cpp t   計測（-O2 でビルドして ./a < in を実行、時間と最大メモリ）

f=${1:-a.cpp}
o=$(basename "$f" .cpp)
S="-std=gnu++20"
ulimit -s 65532 2> /dev/null

case "$2" in
r) g++ $S -O2 -o "$o" "$f" ;;
t) g++ $S -O2 -o "$o" "$f" || exit
   python3 -c '
import subprocess, resource, sys, time, platform
t = time.time()
p = subprocess.run(["./" + sys.argv[1]], stdin=open("in"))
el = time.time() - t
r = resource.getrusage(resource.RUSAGE_CHILDREN).ru_maxrss
mb = r / 1048576 if platform.system() == "Darwin" else r / 1024
print("時間 %.3f s   最大メモリ %.1f MB" % (el, mb),
file=sys.stderr)
sys.exit(p.returncode)' "$o" ;;
*) g++ $S -g -O1 -fsanitize=address,undefined -fno-sanitize-recover=all \
      -Wall -Wextra -Wshadow -o "$o" "$f" ;;
esac
```

gen.cpp

md5: 1c88b3

```cpp
// 乱数で入力を1つ出すだけ
// stress_test.sh から使う。seed は毎回変わる
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
mt19937_64 rng(random_device{}());
ll rnd(ll l, ll r) { return l + rng() % (r - l + 1); }  // [l,r]
int main() {
   ll n = rnd(1, 5);  // ここから下だけ問題ごとに書く
   cout << n << "\n";
```

```
   for(ll i = 0; i < n; i++) cout << rnd(-4, 4) << " ";
}
```

hash.sh

```
# 使い方:
#   sh hash.sh          コピペ -> Ctrl + D
# でその場のコードをハッシュ
#   sh hash.sh --all    src/ 以下を全部ハッシュして
# src/template/hash_values.txt を作り直す
#
# 本番で打つのはこれだけでよい（手で打つコードにコメントは無いの
# で）:
#   tr -d '[:space:]' | md5sum | cut -c-6

MD5() { command -v md5sum > /dev/null 2>&1 && md5sum || md5; }
# コメント・空白・改行を削除して md5。コンパイラに依存しない
H() {
   perl -0777 -pe 's{//[^\n]*|/\*.*?\*/}{}gs' |
      tr -d '[:space:]' | MD5 | cut -c-6
}

if [ "$1" != "--all" ]; then H; exit; fi

R=$(cd "$(dirname "$0")/../.." && pwd)
O="$R/src/template/hash_values.txt"
{
   echo "# 本番でハッシュを確認するコマンド"
   echo "# （打ったコードにコメントは無いので空白を消すだけでよい）"
   echo "#   tr -d '[:space:]' | md5sum | cut -c-6"
   echo "#   -> 貼り付け -> Ctrl + D"
   echo "# 手元でこの表を作り直すコマンド"
   echo "#   sh src/template/hash.sh --all"
   echo
   cd "$R/src" || exit
   for f in $(find . -name '*.hpp' | sed 's|^\./||' | sort); do
      printf '%-45s %s\n' "$f" "$(H < "$f")"
      # 「ここから下は」より上だけの版
      # （二分探索を打たないとき用）
      M='^ *// ===== ここから下は'
      if grep -q "$M" "$f"; then
         h=$(sed "/$M/,\$d" "$f" | sed '$a\};' | H)
         printf '%-45s %s\n' "$f (前半のみ)" "$h"
      fi
   done
} > "$O"
echo "$O を更新しました"
```

hash_values.txt

```
# 本番でハッシュを確認するコマンド（打ったコードにコメントは無いので、空白
を消すだけでよい）
#   tr -d '[:space:]' | md5sum | cut -c-6
#   -> 貼り付け -> Ctrl + D
# 手元でこの表を作り直すコマンド
#   sh src/template/hash.sh --all

data-structure/2DSegtree.hpp                    6ce929
data-structure/ConvexHullTrick.hpp              84c189
data-structure/FastSet.hpp                      bf4f0b
data-structure/fenwick_tree.hpp                 60d4dd
data-structure/lazy_segtree.hpp                 a09c8e
data-structure/lazy_segtree.hpp (前半のみ) d41d8c
data-structure/LiChao.hpp                       92e522
data-structure/Mosalgo.hpp                      6bff59
data-structure/RollbackUnionFind.hpp            ef8861
data-structure/segtree.hpp                      051c9f
data-structure/segtree.hpp (前半のみ)     d41d8c
data-structure/UnionFind.hpp                    8f4a82
extra/modint_fast.hpp                           4b6639
flow/bipartite-matching.hpp                     845322
flow/maxflow.hpp                                0459fb
flow/mincostflow.hpp                            eddc72
FPS/FFT_fast.hpp                                d932d5
FPS/FFT.hpp                                     63d23d
graph/centroid_decomposition.hpp                dae9cb
graph/dijkstra.hpp                              e9adaf
graph/edge.hpp                                  863cfa
graph/eulertrail.hpp                            83e15f
graph/hld.hpp                                   dbcdf8
graph/lowlink.hpp                               07c344
graph/matrix_tree.hpp                           5ce1b7
graph/scc.hpp                                   9d4785
graph/TreeHash.hpp                              c07d91
graph/Trie.hpp                                  63ded2
math/BinaryGCD.hpp                              3ed6fb
math/EulerPhi.hpp                               2443cb
math/ExtGCD.hpp                                 c3fa9b
math/floor_sum.hpp                              930ca0
math/lagrange.hpp                               c96283
math/linear_congruence.hpp                      c6a31e
math/math_template.hpp                          8743ef
math/Matrix.hpp                                 fe12c1
math/primitive_root.hpp                         19c243
math/xorConvolution.hpp                         b6be6f
memo/Bi-Connected-Components.hpp                eddd4d
memo/dynamic_segtree.hpp                        279c02
memo/geometry.hpp                               49dca1
memo/int128.hpp                                 a4c180
memo/random.hpp                                 bf85b5
memo/range_edge_graph.hpp                       f3f96c
modint/BarrettReduction.hpp                     b6414f
modint/combination.hpp                          7f04ef
modint/convolution.hpp                          673c1c
modint/modint.hpp                               3db9f2
string/KMP.hpp                                  8f3c46
string/lcp_array.hpp                            2c8f78
string/Manacher.hpp                             5882fb
string/palindrome.hpp                           c3ad5a
string/rolling_hash.hpp                         830eaf
string/suffix_array.hpp                         79d0c6
string/z_algorithm.hpp                          146991
template/template.hpp                           c48eb9
```

stress_test.sh

```
# cf.cpp(解法) / naive.cpp(愚直) / gen.cpp(乱数入力) を置いて
#   sh stress_test.sh
# 出力の食い違いだけでなく、範囲外アクセス・UB・異常終了でも
# 止まる。ASan の出力はそのまま画面に出る
S="-std=gnu++20 -g -O1 -Wall -Wextra -Wshadow"
A="-fsanitize=address,undefined -fno-sanitize-recover=all"
for f in cf naive gen; do g++ $S $A -o $f $f.cpp || exit; done
ulimit -s 65532 2> /dev/null
i=0
while ./gen > in; do
   i=$((i + 1))
   ./cf < in > o1 && ./naive < in > o2 && cmp -s o1 o2 || break
done
echo "case $i"; cat in o1 o2
```

template.hpp

md5: c48eb9

```
// https://github.com/akuato/icpclibrary
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define rep(i, n) for(ll i = 0; i < (n); i++)
#define repi(i, a, b) for(ll i = (a); i < (b); i++)
#define all(x) (x).begin(), (x).end()
#define arr(x) (x).rbegin(), (x).rend()
template<typename T, typename S> bool chmin(T& a, const S& b) {
   return a > b ? a = b, 1 : 0;
}
template<typename T, typename S> bool chmax(T& a, const S& b) {
   return a < b ? a = b, 1 : 0;
}
using vi = vector<int>;
using P = pair<ll, ll>;
using vvi = vector<vi>;
using vll = vector<ll>;
using vvll = vector<vll>;
using vp = vector<P>;
using i128 = __int128_t;
#define sz(a) (ll)(size(a))
const ll inf = 2e18;

void solve(int test) {}
int main() {
   cin.tie(0);
   ios::sync_with_stdio(false);
   int t = 1;  // cin >>t;
   rep(test, t) solve(test);
}
```

vim_settings.sh

```
sy on
se nu bs=2 cin sta ts=4 sw=4 et ic scs hls is cb=unnamed
let g:loaded_matchparen=1
```

```
ino ( ()<Left>
ino { {}<Left>
ino [ []<Left>
ino <expr> ) getline('.')[col('.')-1]==')'?'<Right>':')'
ino <expr> } getline('.')[col('.')-1]=='}'?'<Right>':'}'
ino <expr> ] getline('.')[col('.')-1]==']'?'<Right>':']'
nn <Space> :noh<CR>
```

data-structure

2DSegtree.hpp

md5: 6ce929

```cpp
#pragma once

template<typename T, typename F> struct SegmentTree2D {
   private:
   int id(int h, int w) const { return h * 2 * W + w; }

   public:
   int H, W;
   vector<T> seg;
   const F f;
   const T I;

   SegmentTree2D(int h, int w, F _f, const T& i) : f(_f), I(i)
{
      init(h, w);
   }

   void init(int h, int w) {
      H = W = 1;
      while(H < h) H <<= 1;
      while(W < w) W <<= 1;
      seg.assign(4 * H * W, I);
   }

   // build にのみ呼ぶ
   void set(int h, int w, const T& x) {
      seg[id(h + H, w + W)] = x;
   }

   void build() {
      // w in [W, 2W)
      for(int w = W; w < 2 * W; w++) {
         for(int h = H - 1; h; h--) {
            seg[id(h, w)] = f(seg[id(2 * h + 0, w)],
                              seg[id(2 * h + 1, w)]);
         }
      }
      // h in [0, 2H)
      for(int h = 0; h < 2 * H; h++) {
         for(int w = W - 1; w; w--) {
            seg[id(h, w)] = f(seg[id(h, 2 * w + 0)],
                              seg[id(h, 2 * w + 1)]);
         }
      }
   }

   T get(int h, int w) const { return seg[id(h + H, w + W)]; }
   T operator()(int h, int w) const {
      return seg[id(h + H, w + W)];
   }

   void update(int h, int w, const T& x) {
      h += H, w += W;
      seg[id(h, w)] = x;
      for(int i = h >> 1; i; i >>= 1) {
         seg[id(i, w)] = f(seg[id(2 * i + 0, w)],
                           seg[id(2 * i + 1, w)]);
      }
      for(; h; h >>= 1) {
         for(int j = w >> 1; j; j >>= 1) {
            seg[id(h, j)] = f(seg[id(h, 2 * j + 0)],
                              seg[id(h, 2 * j + 1)]);
         }
      }
   }

   T _inner_query(int h, int w1, int w2) {
      T res = I;
      for(; w1 < w2; w1 >>= 1, w2 >>= 1) {
         if(w1 & 1) res = f(res, seg[id(h, w1)]), w1++;
         if(w2 & 1) --w2, res = f(res, seg[id(h, w2)]);
      }
      return res;
   }

   // [ (h1,w1), (h2,w2) ) 半開
   T query(int h1, int w1, int h2, int w2) {
      if(h1 >= h2 || w1 >= w2) return I;
      T res = I;
      h1 += H, h2 += H, w1 += W, w2 += W;
      for(; h1 < h2; h1 >>= 1, h2 >>= 1) {
         if(h1 & 1)
            res = f(res, _inner_query(h1, w1, w2)), h1++;
         if(h2 & 1)
            --h2, res = f(res, _inner_query(h2, w1, w2));
      }
      return res;
   }
};

/**
 * @brief 二次元セグメント木
 * @docs docs/data-structure-2d/2d-segment-tree.md
 */

// 使い方
int main() {
   cin.tie(0);
   ios::sync_with_stdio(false);
   int h, w, h1, w1, h2, w2;
   cin >> h >> w >> h1 >> w1 >> h2 >> w2;
   chmin(h2, h1);
   chmin(w2, w1);
   SegmentTree2D seg(
      h + 1, w + 1, [](ll a, ll b) { return max(a, b); },
0ll);
   vvll a(h + 1, vll(w + 1));
   repi(i, 1, h + 1) repi(j, 1, w + 1) cin >> a[i][j];
   vvll sum(h + 1, vll(w + 1)), sumt(h + 1, vll(w + 1)),
      suma(h + 1, vll(w + 1));
   repi(i, 1, h + 1) repi(j, 1, w + 1)
      sum[i][j] = sum[i][j - 1] + a[i][j];
   repi(j, 1, w + 1) repi(i, 1, h + 1)
      sum[i][j] += sum[i - 1][j];
   repi(i, h1, h + 1) repi(j, w1, w + 1)
      sumt[i][j] = sum[i][j] - sum[i - h1][j] - sum[i][j - w1]
                   + sum[i - h1][j - w1];
   repi(i, h2, h + 1) repi(j, w2, w + 1)
      suma[i][j] = sum[i][j] - sum[i - h2][j] - sum[i][j - w2]
                   + sum[i - h2][j - w2];
   repi(i, 1, h + 1) repi(j, 1, w + 1)
      seg.set(i, j, suma[i][j]);
   seg.build();
   ll ans = -inf;
   repi(i, h1, h + 1) repi(j, w1, w + 1) {
      ll now = sumt[i][j];
      now -= seg.query(i - (h1 - h2), j - (w1 - w2), i + 1,
                       j + 1);
      chmax(ans, now);
   }
   cout << ans << endl;
}
```

ConvexHullTrick.hpp

md5: 84c189

```cpp
// https://ei1333.github.io/library/structure/convex-hull-
trick/convex-hull-trick-add-monotone.hpp.html
template<typename T, bool isMin>
struct ConvexHullTrickAddMonotone {
#define F first
#define S second
   using P = pair<T, T>;
   deque<P> H;

   ConvexHullTrickAddMonotone() = default;

   bool empty() const { return H.empty(); }

   void clear() { H.clear(); }

   static constexpr int sgn(T x) {
      return x == 0 ? 0 : (x < 0 ? -1 : 1);
   }

   static constexpr T floor_div(T n, T d) {
      return n / d - ((n ^ d) < 0 and n % d != 0);
   }
```

```
    static constexpr bool check(const P& a, const P& b,
                                const P& c) {
        if(b.S == a.S || c.S == b.S)
            return sgn(b.F - a.F) * sgn(c.S - b.S)
                   >= sgn(c.F - b.F) * sgn(b.S - a.S);
        // return (b.F-a.F)*(c.S-b.S) >= (b.S-a.S)*(c.F-b.F);
        if constexpr(is_integral<T>::value) {
            return floor_div(b.S - a.S, a.F - b.F)
                   >= floor_div(c.S - b.S, b.F - c.F);
        } else {
            return (b.F - a.F) * sgn(c.S - b.S) / abs(b.S - a.S)
                   >= (c.F - b.F) * sgn(b.S - a.S)
                          / abs(c.S - b.S);
        }
    }

    void add(T a, T b) {
        if(!isMin) a *= -1, b *= -1;
        P line(a, b);
        if(empty()) {
            H.emplace_front(line);
            return;
        }
        if(H.front().F <= a) {
            if(H.front().F == a) {
                if(H.front().S <= b) return;
                H.pop_front();
            }
            while(H.size() >= 2 && check(line, H.front(), H[1]))
                H.pop_front();
            H.emplace_front(line);
        } else {
            assert(a <= H.back().F);
            if(H.back().F == a) {
                if(H.back().S <= b) return;
                H.pop_back();
            }
            while(H.size() >= 2
                  && check(H[H.size() - 2], H.back(), line))
                H.pop_back();
            H.emplace_back(line);
        }
    }

    static constexpr T get_y(const P& a, const T& x) {
        return a.F * x + a.S;
    }

    T query(T x) const {
        assert(!empty());
        int l = -1, r = H.size() - 1;
        while(l + 1 < r) {
            int m = (l + r) >> 1;
            if(get_y(H[m], x) >= get_y(H[m + 1], x)) l = m;
            else r = m;
        }
        if(isMin) return get_y(H[r], x);
        return -get_y(H[r], x);
    }

    T query_monotone_inc(T x) {
        assert(!empty());
        while(H.size() >= 2
              && get_y(H.front(), x) >= get_y(H[1], x))
            H.pop_front();
        if(isMin) return get_y(H.front(), x);
        return -get_y(H.front(), x);
    }

    T query_monotone_dec(T x) {
        assert(!empty());
        while(H.size() >= 2
              && get_y(H.back(), x) >= get_y(H[H.size() - 2], x))
            H.pop_back();
        if(isMin) return get_y(H.back(), x);
        return -get_y(H.back(), x);
    }

#undef F
#undef S
};
```

FastSet.hpp

md5: bf4f0b

```
using u64 = uint64_t;

const u64 B = 64;
struct FastSet {
    u64 n;
    vector<vector<u64>> a;
    FastSet(u64 n_) : n(n_) {
        do a.emplace_back(n_ = (n_ + B - 1) / B);
        while(n_ > 1);
    }
    // bool operator[](ll i) const {
    //     return a[0][i / B] >> (i % B) & 1;
    // }
    void set(ll i) {
        for(auto& v : a) {
            v[i / B] |= 1ULL << (i % B);
            i /= B;
        }
    }
    void reset(ll i) {
        for(auto& v : a) {
            v[i / B] &= ~(1ULL << (i % B));
            if(v[i / B]) break;
            i /= B;
        }
    }
    ll next(ll i) {  // i を超える最小の要素
        repi(h, 0, sz(a)) {
            i++;
            if(i / B >= sz(a[h])) break;
            u64 d = a[h][i / B] >> (i % B);
            if(d) {
                i += countr_zero(d);
                while(h--) i = i * B + countr_zero(a[h][i]);
                return i;
            }
            i /= B;
        }
        return n;
    }
    ll prev(ll i) {  // i より小さい最大の要素
        repi(h, 0, sz(a)) {
            i--;
            if(i < 0) break;
            u64 d = a[h][i / B] << (~i % B);
            if(d) {
                i -= countl_zero(d);
                while(h--) i = i * B + __lg(a[h][i]);
                return i;
            }
            i /= B;
        }
        return -1;
    }
};
```

LiChao.hpp

md5: 831efe

```
// 直線 ax+b を足して、x での最小値を返す
// 最大値が欲しいときは LiChao<false>
//
// 要件
//   傾きの単調性は不要
//   query する x の候補を昇順・重複なしで、構築時に全部渡す
//
// 使い方
//   vll xs = {...};        昇順・重複なし
//   sort(all(xs));
//   xs.erase(unique(all(xs)), xs.end());
//   LiChao<true> c(xs);
//   c.add(a, b);           直線 ax+b を足す
//   c.query(x);            x は xs の値（assert で確認）
//
// add も query も O(log n)
// 傾きが単調なら cht / cht_monotone の方が軽い
template<bool isMin = true> struct LiChao {
    vll xs;
    vp t;
    LiChao(vll xs) : xs(xs), t(4 * sz(xs), P(0, inf)) {}
    ll f(P l, ll x) { return l.first * x + l.second; }
    void add(ll a, ll b) {
        if(!isMin) a = -a, b = -b;
        P l(a, b);
```

```
      int k = 1, lo = 0, hi = sz(xs) - 1;
      while(1) {
         int m = (lo + hi) / 2;
         bool bl = f(l, xs[lo]) < f(t[k], xs[lo]);
         bool bm = f(l, xs[m]) < f(t[k], xs[m]);
         if(bm) swap(t[k], l);
         if(lo == hi) return;
         if(bl != bm) hi = m, k = 2 * k;
         else lo = m + 1, k = 2 * k + 1;
      }
   }
   ll query(ll x) {  // x は xs に含まれる値
      int k = 1, lo = 0, hi = sz(xs) - 1;
      ll res = inf;
      while(1) {
         chmin(res, f(t[k], x));
         if(lo == hi) {
            assert(xs[lo] == x);  // x は xs に無いと答えが狂う
            return isMin ? res : -res;
         }
         int m = (lo + hi) / 2;
         if(x <= xs[m]) hi = m, k = 2 * k;
         else lo = m + 1, k = 2 * k + 1;
      }
   }
};
```

LiChaoDyn.hpp

md5: 699ec1

```
// 直線 ax+b を足して、x での最小値を返す
// 最大値が欲しいときは LiChaoDyn<false>
//
// 要件
//   傾きの単調性は不要。x の候補も事前に要らない
//   x のとりうる範囲 [lo, hi]（両端含む）だけ指定する
//
// 使い方
//   LiChaoDyn<true> c(-1000000000, 1000000000);
//   c.add(a, b);     直線 ax+b を足す
//   c.query(x);      最小値。1本も無ければ inf（最大版は -inf）
//
// add も query も O(log(hi-lo))
// ノードは必要な分だけ作る。1ノード 24 バイト
// 最悪 O(n log(hi-lo)) だが経路が重なるので実際は遥かに少ない
// 範囲 2*10^9 に 20 万本足して数千ノード（0.1 MB）だった
template<bool isMin = true> struct LiChaoDyn {
   vp t;
   vi lc, rc;
   ll L, R;
   LiChaoDyn(ll L, ll R) : L(L), R(R) { nw(); }
   int nw() {
      t.push_back(P(0, inf));
      lc.push_back(-1);
      rc.push_back(-1);
      return sz(t) - 1;
   }
   ll f(P l, ll x) { return l.first * x + l.second; }
   void add(ll a, ll b) {
      if(!isMin) a = -a, b = -b;
      P l(a, b);
      int k = 0;
      ll lo = L, hi = R;
      while(1) {
         ll m = lo + (hi - lo) / 2;
         bool bl = f(l, lo) < f(t[k], lo);
         bool bm = f(l, m) < f(t[k], m);
         if(bm) swap(t[k], l);
         if(lo == hi) return;
         if(bl != bm) {
            if(lc[k] < 0) lc[k] = nw();
            k = lc[k], hi = m;
         } else {
            if(rc[k] < 0) rc[k] = nw();
            k = rc[k], lo = m + 1;
         }
      }
   }
   ll query(ll x) {
      assert(L <= x && x <= R);
      int k = 0;
      ll lo = L, hi = R, res = inf;
      while(k >= 0) {
         chmin(res, f(t[k], x));
         if(lo == hi) break;
         ll m = lo + (hi - lo) / 2;
         if(x <= m) k = lc[k], hi = m;
         else k = rc[k], lo = m + 1;
      }
      return isMin ? res : -res;
   }
};
```

Mosalgo.hpp

md5: 6bff59

```
int n;                    // 数列の長さ
int q;                    // クエリ数
vector<int> l(q), r(q);   // クエリの[l,r)
/* 入力 */

// ソート
int sq = max(1, int(n / sqrt(q)));
vector<int> qi(q);
for(int i = 0; i < q; ++i) qi[i] = i;
sort(qi.begin(), qi.end(),
     [&](int i, int j) {  // クエリの順番をpair(l/sq,r)でソート
        if(l[i] / sq != l[j] / sq) return l[i] < l[j];
        return r[i] < r[j];
     });

// クエリの処理
// add(i)は追加操作、del(i)は削除操作
int nl = 0, nr = 0;  // [nl,nr)
for(int i : qi) {
   while(nl > l[i]) --nl, add(nl);
   while(nr < r[i]) add(nr), ++nr;
   while(nl < l[i]) del(nl), ++nl;
   while(nr > r[i]) --nr, del(nr);
   /* クエリの結果計算 */
}
```

RollbackUnionFind.hpp

md5: ef8861

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

struct RollbackUnionFind {
   vector<int> data;
   stack<pair<int, int>> history;
   int inner_snap = 0;
   RollbackUnionFind(int n) { data.resize(n, -1); }
   int find(int x) { return data[x] < 0 ? x : find(data[x]); }
   bool unite(int x, int y) {
      x = find(x), y = find(y);
      history.push({x, data[x]});
      history.push({y, data[y]});
      if(x == y) return false;
      if(-data[x] < -data[y]) swap(x, y);
      data[x] += data[y];
      data[y] = x;
      return true;
   }
   int same(int x, int y) { return find(x) == find(y); }
   int size(int x) { return (-data[find(x)]); }
   void undo() {
      data[history.top().first] = history.top().second;
      history.pop();
      data[history.top().first] = history.top().second;
      history.pop();
   }
   int time() { return int(history.size() >> 1); }
   void snapshot() { inner_snap = time(); }
   void rollback(int t = -1) {
      if(t == -1) t = inner_snap;
      while(t < time()) undo();
   }
};
```

UnionFind.hpp

md5: 8f4a82

```
struct UnionFind {
   vector<int> par, siz;
   UnionFind(int n) : par(n, -1), siz(n, 1) {}
   int root(int x) {
      if(par[x] == -1) return x;
      else return par[x] = root(par[x]);
   }
   bool issame(int x, int y) { return root(x) == root(y); }
   bool unite(int x, int y) {
      x = root(x), y = root(y);
```

```
      if(x == y) return false;
      if(siz[x] < siz[y]) swap(x, y);
      par[y] = x;
      siz[x] += siz[y];
      return true;
   }
   int size(int x) { return siz[root(x)]; }
};
```

cht.hpp

md5: a101c0

```
// 直線 ax+b を足して、任意の x での最小値を返す
// 最大値が欲しいときは CHT<false>
//
// 要件
//   add する傾き a は減少のみ（最大版 <false> なら増加のみ）
//   query の x は任意の順でよい
//
// 傾きが逆向きのときは x を反転する
//   a*x + b = (-a)*(-x) + b
//   傾き増加なら add(-a, b) として query(-x) を呼ぶ
//
// 使い方
//   CHT<true> c;
//   c.add(a, b);     直線 ax+b を足す
//   c.query(x);      最小値。1本も無ければ inf（最大版は -inf）
//
// add 償却 O(1)、query O(log n)
// x も単調なら cht_monotone（query も O(1)）
template<bool isMin = true> struct CHT {
   vp d;
   bool bad(P x, P y, P z) {
      return (i128)(z.second - x.second) * (x.first - y.first)
            <= (i128)(y.second - x.second)
                  * (x.first - z.first);
   }
   ll val(P l, ll x) { return l.first * x + l.second; }
   void add(ll a, ll b) {
      if(!isMin) a = -a, b = -b;
      if(!d.empty()) assert(a <= d.back().first);
      P l(a, b);
      if(!d.empty() && d.back().first == a) {
         if(d.back().second <= b) return;
         d.pop_back();
      }
      while(sz(d) >= 2 && bad(d[sz(d) - 2], d[sz(d) - 1], l))
         d.pop_back();
      d.push_back(l);
   }
   ll query(ll x) {
      if(d.empty()) return isMin ? inf : -inf;
      int lo = 0, hi = sz(d) - 1;
      while(lo < hi) {
         int m = (lo + hi) / 2;
         if(val(d[m], x) >= val(d[m + 1], x)) lo = m + 1;
         else hi = m;
      }
      return isMin ? val(d[lo], x) : -val(d[lo], x);
   }
};
```

cht_light.hpp

md5: 4ce994

```
// 直線 ax+b を足して、ある x での最小値を返す
// 最大値が欲しいときは CHTLight<false>
//
// 要件
//   add する傾き a は、最小版 <true> なら減少のみ
//                      最大版 <false> なら増加のみ
//   query の x は単調。増加なら qinc、減少なら qdec
//   qinc と qdec は混ぜない
//
// 使い方
//   CHTLight<true> c;
//   c.add(a, b);   直線 ax+b を足す
//   c.qinc(x);     x が増えていくとき
//   c.qdec(x);     x が減っていくとき
//   1本も無ければ inf（最大版は -inf）
//
// add も query も償却 O(1)
// deque ではなく vector なので、空なら確保ゼロ
// deque は空でも約 656 バイト使う。10^5 個以上を並べて
// 持つときはこちら（100万個で 656 MB 対 30 MB）
// 捨てた直線のメモリは戻らない。使用量は「足した総数」になる
template<bool isMin = true> struct CHTLight {
   vp d;
   int h = 0;  // 生きている先頭
   bool bad(P x, P y, P z) {
      return (i128)(z.second - x.second) * (x.first - y.first)
            <= (i128)(y.second - x.second)
                  * (x.first - z.first);
   }
   ll val(P l, ll x) { return l.first * x + l.second; }
   void add(ll a, ll b) {
      if(!isMin) a = -a, b = -b;
      if(!d.empty()) assert(a <= d.back().first);
      P l(a, b);
      if(sz(d) > h && d.back().first == a) {
         if(d.back().second <= b) return;
         d.pop_back();
      }
      while(sz(d) - h >= 2
            && bad(d[sz(d) - 2], d[sz(d) - 1], l))
         d.pop_back();
      d.push_back(l);
   }
   ll qinc(ll x) {  // x が増えていくとき
      if(h >= sz(d)) return isMin ? inf : -inf;
      while(sz(d) - h >= 2 && val(d[h], x) >= val(d[h + 1], x))
         h++;
      return isMin ? val(d[h], x) : -val(d[h], x);
   }
   ll qdec(ll x) {  // x が減っていくとき
      if(h >= sz(d)) return isMin ? inf : -inf;
      while(sz(d) - h >= 2
            && val(d[sz(d) - 1], x) >= val(d[sz(d) - 2], x))
         d.pop_back();
      ll v = val(d[sz(d) - 1], x);
      return isMin ? v : -v;
   }
};
```

cht_monotone.hpp

md5: 710ba1

```
// 直線 ax+b を足して、ある x での最小値を返す
// 最大値が欲しいときは CHTMonotone<false>
//
// 要件
//   add する傾き a は減少のみ（最大版 <false> なら増加のみ）
//   query の x も単調。増加なら qinc、減少なら qdec
//   qinc と qdec は混ぜない
//
// 傾きが逆向きのときは x を反転すれば同じことになる
//   a*x + b = (-a)*(-x) + b   なので傾きと x が同時に反転する
//   最小版での対応（最大版は「減少」と「増加」を入れ替え）
//     傾き減少 x増加 -> add(a, b)   qinc(x)
//     傾き減少 x減少 -> add(a, b)   qdec(x)
//     傾き増加 x減少 -> add(-a, b)  qinc(-x)
//     傾き増加 x増加 -> add(-a, b)  qdec(-x)
//
// 使い方
//   CHTMonotone<true> c;
//   c.add(a, b);   直線 ax+b を足す
//   c.qinc(x);     x が増えていくとき
//   c.qdec(x);     x が減っていくとき
//   1本も無ければ inf（最大版は -inf）
//
// add も query も償却 O(1)
// vector なので空なら確保ゼロ。10^5 個以上並べて持てる
// qinc は捨てた直線のメモリが戻らない（使用量は足した総数）
template<bool isMin = true> struct CHTMonotone {
   vp d;
   int h = 0;  // 生きている先頭
   bool bad(P x, P y, P z) {
      return (i128)(z.second - x.second) * (x.first - y.first)
            <= (i128)(y.second - x.second)
                  * (x.first - z.first);
   }
   ll val(P l, ll x) { return l.first * x + l.second; }
   void add(ll a, ll b) {
      if(!isMin) a = -a, b = -b;
      if(!d.empty()) assert(a <= d.back().first);
      P l(a, b);
      if(sz(d) > h && d.back().first == a) {
         if(d.back().second <= b) return;
         d.pop_back();
      }
      while(sz(d) - h >= 2
            && bad(d[sz(d) - 2], d[sz(d) - 1], l))
         d.pop_back();
```

```
      d.push_back(l);
   }
   ll qinc(ll x) {  // x が増えていくとき
      if(h >= sz(d)) return isMin ? inf : -inf;
      while(sz(d) - h >= 2 && val(d[h], x) >= val(d[h + 1], x))
         h++;
      return isMin ? val(d[h], x) : -val(d[h], x);
   }
   ll qdec(ll x) {  // x が減っていくとき
      if(h >= sz(d)) return isMin ? inf : -inf;
      while(sz(d) - h >= 2
            && val(d[sz(d) - 1], x) >= val(d[sz(d) - 2], x))
         d.pop_back();
      ll v = val(d[sz(d) - 1], x);
      return isMin ? v : -v;
   }
};
```

fenwick_tree.hpp

md5: 60d4dd

```
struct fenwick_tree {
   vector<ll> a;
   fenwick_tree(ll n) : a(n + 1) {}
   void add(ll i, ll x) {  // A[i] += x
      i++;
      while(i < sz(a)) {
         a[i] += x;
         i += i & -i;
      }
   }
   ll sum(ll r) {
      ll s = 0;
      while(r) {
         s += a[r];
         r -= r & -r;
      }
      return s;
   }
   ll sum(ll l, ll r) {  // sum of A[l, r)
      return sum(r) - sum(l);
   }
   // sum(i+1) >= x となる最小の i。無ければ n
   // A[i] >= 0 が前提
   ll lower_bound(ll x) {
      ll i = 0, n = sz(a) - 1, k = 1;
      while(k * 2 <= n) k *= 2;
      for(; k; k /= 2)
         if(i + k <= n && a[i + k] < x) i += k, x -= a[i];
      return i;
   }
};
```

lazy_segtree.hpp

md5: a09c8e

```
// https://github.com/YNUCPC/ynu-icpc-
library/blob/main/src/datastructure/lazysegtree.hpp
template<class S, S (*op)(S, S), S (*e)(), class F,
         S (*mapping)(F, S), F (*composition)(F, F), F (*id)()>
struct lazy_segtree {  // ACL と同じ名前・同じ使い方
   int _n, size, log;
   vector<S> dat;
   vector<F> lz;
   void update(int k) {
      dat[k] = op(dat[2 * k], dat[2 * k + 1]);
   }
   void all_apply(int k, F f) {
      dat[k] = mapping(f, dat[k]);
      if(k < size) lz[k] = composition(f, lz[k]);
   }
   void push(int k) {
      all_apply(2 * k, lz[k]);
      all_apply(2 * k + 1, lz[k]);
      lz[k] = id();
   }
   int lower_bits(int x, int k) { return x & ((1 << k) - 1); }

   lazy_segtree() : lazy_segtree(0) {}
   lazy_segtree(int n) : lazy_segtree(vector<S>(n, e())) {}
   lazy_segtree(const vector<S>& v) : _n(int(v.size())) {
      log = 0;
      while((1 << log) < _n) log++;
      size = 1 << log;
      dat = vector<S>(2 * size, e());
      lz = vector<F>(size, id());
      for(int i = 0; i < _n; i++) dat[size + i] = v[i];
      for(int i = size - 1; i >= 1; i--) update(i);
   }
   // a[p] = x
   void set(int p, S x) {
      p += size;
      for(int i = log; i >= 1; i--) push(p >> i);
      dat[p] = x;
      for(int i = 1; i <= log; i++) update(p >> i);
   }
   // return a[p]
   S get(int p) {
      p += size;
      for(int i = log; i >= 1; i--) push(p >> i);
      return dat[p];
   }
   // return op(a[l], ..., a[r-1])
   S prod(int l, int r) {
      if(l == r) return e();
      l += size;
      r += size;
      for(int i = log; i >= 1; i--) {
         if(lower_bits(l, i) > 0) push(l >> i);
         if(lower_bits(r, i) > 0) push((r - 1) >> i);
      }
      S sml = e(), smr = e();
      while(l < r) {
         if(l & 1) sml = op(sml, dat[l++]);
         if(r & 1) smr = op(dat[--r], smr);
         l >>= 1;
         r >>= 1;
      }
      return op(sml, smr);
   }
   S all_prod() { return dat[1]; }
   // a[p] = f(a[p])
   void apply(int p, F f) {
      p += size;
      for(int i = log; i >= 1; i--) push(p >> i);
      dat[p] = mapping(f, dat[p]);
      for(int i = 1; i <= log; i++) update(p >> i);
   }
   // i = l...r-1 について a[i] = f(a[i])
   void apply(int l, int r, F f) {
      if(l == r) return;
      l += size;
      r += size;
      for(int i = log; i >= 1; i--) {
         if(lower_bits(l, i) > 0) push(l >> i);
         if(lower_bits(r, i) > 0) push((r - 1) >> i);
      }
      int l2 = l, r2 = r;
      while(l < r) {
         if(l & 1) all_apply(l++, f);
         if(r & 1) all_apply(--r, f);
         l >>= 1;
         r >>= 1;
      }
      l = l2;
      r = r2;
      for(int i = 1; i <= log; i++) {
         if(lower_bits(l, i) > 0) update(l >> i);
         if(lower_bits(r, i) > 0) update((r - 1) >> i);
      }
   }
   // ===== ここから下は木上の二分探索。要らなければ打たない
   // f(op(a[l], ..., a[r-1])) == true となる最大の r
   template<bool (*g)(S)> int max_right(int l) {
      return max_right(l, [](S x) { return g(x); });
   }
   template<class G> int max_right(int l, G g) {
      assert(g(e()));
      if(l == _n) return _n;
      l += size;
      for(int i = log; i >= 1; i--) push(l >> i);
      S sm = e();
      do {
         while(l % 2 == 0) l >>= 1;
         if(!g(op(sm, dat[l]))) {
            while(l < size) {
               push(l);
               l = (2 * l);
               if(g(op(sm, dat[l]))) {
                  sm = op(sm, dat[l]);
                  l++;
               }
            }
```

```
            return l - size;
        }
        sm = op(sm, dat[l]);
        l++;
    } while((l & -l) != l);
    return _n;
}
// f(op(a[l], ..., a[r-1])) == true となる最小の l
template<bool (*g)(S)> int min_left(int r) {
    return min_left(r, [](S x) { return g(x); });
}
template<class G> int min_left(int r, G g) {
    assert(g(e()));
    if(r == 0) return 0;
    r += size;
    for(int i = log; i >= 1; i--) push((r - 1) >> i);
    S sm = e();
    do {
        r--;
        while(r > 1 && (r % 2)) r >>= 1;
        if(!g(op(dat[r], sm))) {
            while(r < size) {
                push(r);
                r = (2 * r + 1);
                if(g(op(dat[r], sm))) {
                    sm = op(dat[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(dat[r], sm);
    } while((r & -r) != r);
    return 0;
}
};
```

segtree.hpp

md5: 051c9f

```
// https://github.com/YNUCPC/ynu-icpc-library/blob/main/src/datastructure/segtree.hpp
// ACL と同じ名前・同じ使い方
// segtree<S, op, e> seg(n) または seg(v)
template<class S, S (*op)(S, S), S (*e)()> struct segtree {
    int _n, size, log;
    vector<S> dat;
    void update(int k) {
        dat[k] = op(dat[2 * k], dat[2 * k + 1]);
    }
    segtree() : segtree(0) {}
    segtree(int n) : segtree(vector<S>(n, e())) {}
    segtree(const vector<S>& v) : _n(int(v.size())) {
        log = 0;
        while((1 << log) < _n) log++;
        size = 1 << log;
        dat = vector<S>(2 * size, e());
        for(int i = 0; i < _n; i++) dat[size + i] = v[i];
        for(int i = size - 1; i >= 1; i--) update(i);
    }
    // a[p] = x
    void set(int p, S x) {
        p += size;
        dat[p] = x;
        for(int i = 1; i <= log; i++) update(p >> i);
    }
    S get(int p) const { return dat[p + size]; }
    // op(a[l], ..., a[r-1])
    S prod(int l, int r) const {
        S sml = e(), smr = e();
        l += size, r += size;
        while(l < r) {
            if(l & 1) sml = op(sml, dat[l++]);
            if(r & 1) smr = op(dat[--r], smr);
            l >>= 1, r >>= 1;
        }
        return op(sml, smr);
    }
    S all_prod() const { return dat[1]; }

    // ===== ここから下は木上の二分探索。要らなければ打たない
    // f(op(a[l], ..., a[r-1])) == true となる最大の r
    template<bool (*f)(S)> int max_right(int l) const {
        return max_right(l, [](S x) { return f(x); });
    }
    template<class F> int max_right(int l, F f) const {
        assert(f(e()));
        if(l == _n) return _n;
        l += size;
        S sm = e();
        do {
            while(l % 2 == 0) l >>= 1;
            if(!f(op(sm, dat[l]))) {
                while(l < size) {
                    l = 2 * l;
                    if(f(op(sm, dat[l]))) sm = op(sm, dat[l]), l++;
                }
                return l - size;
            }
            sm = op(sm, dat[l]), l++;
        } while((l & -l) != l);
        return _n;
    }
    // f(op(a[l], ..., a[r-1])) == true となる最小の l
    template<bool (*f)(S)> int min_left(int r) const {
        return min_left(r, [](S x) { return f(x); });
    }
    template<class F> int min_left(int r, F f) const {
        assert(f(e()));
        if(r == 0) return 0;
        r += size;
        S sm = e();
        do {
            r--;
            while(r > 1 && (r % 2)) r >>= 1;
            if(!f(op(dat[r], sm))) {
                while(r < size) {
                    r = 2 * r + 1;
                    if(f(op(dat[r], sm))) sm = op(dat[r], sm), r--;
                }
                return r + 1 - size;
            }
            sm = op(dat[r], sm);
        } while((r & -r) != r);
        return 0;
    }
};
```

math

BinaryGCD.hpp

md5: 3ed6fb

```
using u64 = uint64_t;
u64 ctz(u64 x) { return countr_zero(x); }
u64 binary_gcd(u64 x, u64 y) {
    if(!x || !y) return x | y;
    u64 n = ctz(x), m = ctz(y);
    x >>= n, y >>= m;
    while(x != y) {
        if(x > y) x = (x - y) >> ctz(x - y);
        else y = (y - x) >> ctz(y - x);
    }
    return x << min(n, m);
}
```

EulerPhi.hpp

md5: 2443cb

```
int64_t euler_phi(int64_t n) {
    int64_t ret = n;
    for(int64_t i = 2; i * i <= n; i++) {
        if(n % i == 0) {
            ret -= ret / i;
            while(n % i == 0) n /= i;
        }
    }
    if(n > 1) ret -= ret / n;
    return ret;
}
```

ExtGCD.hpp

md5: c3fa9b

```
// returns gcd(a, b) and assign x, y to integers
// s.t. ax + by = gcd(a, b) and |x| + |y| is minimized
ll extgcd(ll a, ll b, ll& x, ll& y) {
    // assert(a >= 0 && b >= 0);
    if(!b) return x = 1, y = 0, a;
    ll d = extgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

Matrix.hpp

md5: fe12c1

```cpp
// https://youtu.be/ylWYSurx10A?t=2352
template<typename T> struct Matrix {
   int h, w;
   vector<vector<T>> d;
   Matrix() {}
   Matrix(int h, int w, T val = 0)
       : h(h), w(w), d(h, vector<T>(w, val)) {}
   Matrix& unit() {
      // assert(h == w);
      rep(i, h) d[i][i] = 1;
      return *this;
   }
   const vector<T>& operator[](int i) const { return d[i]; }
   vector<T>& operator[](int i) { return d[i]; }
   Matrix operator*(const Matrix& a) const {
      //   assert(w == a.h);
      Matrix r(h, a.w);
      rep(i, h) rep(k, w) rep(j, a.w) {
         r[i][j] += d[i][k] * a[k][j];
      }
      return r;
   }
   Matrix pow(long long t) const {
      //  assert(h == w);
      if(!t) return Matrix(h, h).unit();
      if(t == 1) return *this;
      Matrix r = pow(t >> 1);
      r = r * r;
      if(t & 1) r = r * (*this);
      return r;
   }
   // https://youtu.be/-j02o6__jgs?t=11273
   /* mint only
   mint det() {
     assert(h == w);
     mint res = 1;
     rep(k,h) {
       for (int i = k; i < h; ++i) {
         if (d[i][k] == 0) continue;
         if (i != k) {
           swap(d[i],d[k]);
           res = -res;
         }
       }
       if (d[k][k] == 0) return 0;
       res *= d[k][k];
       mint inv = mint(1)/d[k][k];
       rep(j,h) d[k][j] *= inv;
       for (int i = k+1; i < h; ++i) {
         mint c = d[i][k];
         for (int j = k; j < h; ++j) d[i][j] -= d[k][j]*c;
       }
     }
     return res;
   }
   //*/
};
```

floor_sum.hpp

md5: 930ca0

```cpp
// x_i=floor((a*i+b)/c), i=0,1,..n-1
// a,c>0, b>=0
ll floor_sum(ll n, ll a, ll b, ll c) {
   if(n == 0) return 0;
   ll res = 0;
   res += n * (n - 1) / 2 * (a / c);
   a %= c;
   res += n * (b / c);
   b %= c;
   if(a == 0) return res;
   ll top = (a * (n - 1) + b) / c;
   res += top * n;
   ll h = (b + 1 + c - 1) / c;
   if(h <= top)
      res -= floor_sum(top - h + 1, c, c * h - (b + 1), a) +
top
             - h + 1;
   return res;
}
```

lagrange.hpp

md5: c96283

```cpp
#define FOR(i, a, n) for(ll i = (ll)a; i < (ll)n; ++i)
#define REP(i, n) FOR(i, 0, n)
#define ALL(x) x.begin(), x.end()

// x座標が相異なるn+1点(x_i,y_i)を通るn次以下の多項式f(x)を返す
// O(n^2)
vector<mint> lagrange_interpolation(vector<mint> x,
                                    vector<mint> y) {
   const ll n = x.size() - 1;
   REP(i, n + 1) {
      mint t = 1;
      REP(j, n + 1) if(i != j) t *= x[i] - x[j];
      y[i] /= t;
   }
   ll cur = 0, nxt = 1;
   vector<vector<mint>> dp(2, vector<mint>(n + 2));
   dp[0][0] = -1 * x[0], dp[0][1] = 1;
   FOR(i, 1, n + 1) {
      REP(j, n + 2) {
         dp[nxt][j] = dp[cur][j] * -1 * x[i];
         if(j >= 1) dp[nxt][j] += dp[cur][j - 1];
      }
      swap(nxt, cur);
   }
   vector<mint> xinv(n + 1);
   REP(i, n + 1) xinv[i] = x[i].inv();
   vector<mint> ret(n + 1);  // f(x)
   REP(i, n + 1) {
      if(y[i] == 0) continue;
      // 0割り対策の場合分け
      if(x[i] == 0) {
         REP(j, n + 1) ret[j] += dp[cur][j + 1] * y[i];
      } else {
         ret[0] -= dp[cur][0] * xinv[i] * y[i];
         mint pre = -1 * dp[cur][0] * xinv[i];
         FOR(j, 1, n + 1) {
            ret[j] -= (dp[cur][j] - pre) * xinv[i] * y[i];
            pre = -1 * (dp[cur][j] - pre) * xinv[i];
         }
      }
   }
   return ret;
}

mint lagrange_interpolationT(vector<mint> x, vector<mint> y,
                             mint T) {
   const ll n = x.size() - 1;
   mint ret = 0;
   REP(i, n + 1) {
      mint t = 1;
      REP(j, n + 1) {
         if(i == j) continue;
         t *= T - x[j];
         t /= x[i] - x[j];
      }
      ret += t * y[i];
   }
   return ret;
}
```

linear_congruence.hpp

md5: c6a31e

```cpp
ll extgcd(ll a, ll b, ll& x, ll& y) {
   ll d = a;
   if(b != 0) {
      d = extgcd(b, a % b, y, x);
      y -= (a / b) * x;
   } else {
      x = 1;
      y = 0;
   }
   return d;
}
ll mod_inverse(ll a, ll m) {
   ll x, y;
   extgcd(a, m, x, y);
   return (m + x % m) % m;
}

P linear_congruence(vll& A, vll& B, vll& M) {
   ll x = 0, m = 1;
   rep(i, A.size()) {
      ll a = A[i] * m, b = B[i] - A[i] * x, d = gcd(M[i], a);
```

```
      if(b % d) return P(0, -1);
      ll t = (b / d) * mod_inverse(a / d, M[i] / d)
             % (M[i] / d);
      x = x + m * t;
      m *= M[i] / d;
   }
   return P((x % m + m) % m, m);
}
```

math_template.hpp

md5: 8743ef

```
ll pow_pow(ll x, ll n, ll mod) {
   if(n == 0) return 1;
   x %= mod;
   ll res = pow_pow(x * x % mod, n / 2, mod);
   if(n & 1) res = res * x % mod;
   return res;
}
ll gcdll(ll x, ll y) {
   if(y == 0) return x;
   return gcdll(y, x % y);
}
ll INFF = 1000100100100100100;
ll lcmll(ll x, ll y) {
   ll X = (x / gcdll(x, y)), Y = y;
   if(X <= INFF / Y) return X * Y;
   else return INFF;
}
```

primitive_root.hpp

md5: 19c243

```
template<class Int = long long>
Int find_smallest_primitive_root(Int p) {
   std::vector<Int> fac;
   Int v = p - 1;
   for(Int pp = 2; pp * pp <= v;
       pp++) {  // prime factorization of (p - 1)
      int e = 0;
      while(v % pp == 0) e++, v /= pp;
      if(e) fac.push_back(pp);
   }
   if(v > 1) fac.push_back(v);

   for(Int g = 1; g < p; g++) {
      if(pow_mod(g, p - 1, p) != 1) return -1;
      bool ok = true;
      for(auto pp : fac) {
         if(pow_mod(g, (p - 1) / pp, p) == 1) {
            ok = false;
            break;
         }
      }
      if(ok) return g;
   }
   return -1;
}
```

xorConvolution.hpp

md5: b6be6f

```
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/math/xor-
convolution.hpp
// sizeは２べき
int my_lg(int n) { return 31 - __builtin_clz(n); }
#define si(a) (int)(size(a))
#define fore(e, v) for(auto&& e : v)
template<typename T> void fwt(vector<T>& f, bool inv = false) {
   int n = si(f), m = my_lg(n);
   rep(i, m) {
      rep(b, n) {
         if(~b >> i & 1) {
            T x = f[b], y = f[b ^ 1 << i];
            f[b] = x + y, f[b ^ 1 << i] = x - y;
         }
      }
   }
   if(inv) {
      T iz = T(1) / T(si(f));
      fore(e, f) e *= iz;
   }
}
/*
void solve(int test){
    vm a={1,1,1};
    vm b={1,1,1};
    fwt<mint>(a);
    fwt<mint>(b);
    vm c=a;
    rep(i,c.size())c[i]=a[i]*b[i];
    fwt<mint>(c,true);
}
*/
```

modint

BarrettReduction.hpp

md5: b6414f

```
using u64 = uint64_t;

struct Barrett {  // mod < 2^32
   u64 m, im;
   Barrett(u64 mod) : m(mod), im(-1ULL / m + 1) {}
   // input: a * b < 2^64, output: a * b % mod
   u64 mul(u64 a, u64 b) const {
      a *= b;
      u64 x = ((__uint128_t)a * im) >> 64;
      a -= x * m;
      if((ll)a < 0) a += m;
      return a;
   }
};
```

combination.hpp

md5: 7f04ef

```
constexpr int MAX = 10;
ll fact[MAX], finv[MAX], invv[MAX];
void initcomb() {
   fact[0] = fact[1] = 1;
   finv[0] = finv[1] = 1;
   invv[1] = 1;
   for(int i = 2; i < MAX; i++) {
      fact[i] = fact[i - 1] * i % mod;
      invv[i] = mod - invv[mod % i] * (mod / i) % mod;
      finv[i] = finv[i - 1] * invv[i] % mod;
   }
}
ll comb(ll n, ll k) {
   if(n < k) return 0;
   if(n < 0 || k < 0) return 0;
   return fact[n] * (finv[k] * finv[n - k] % mod) % mod;
}
```

convolution.hpp

md5: 673c1c

```
using vm = vector<mint>;
int my_lg(int n) { return 31 - __builtin_clz(n); }

//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/F
#include <bit>
#define si(a) (int)(size(a))
#define fore(e, v) for(auto&& e : v)
#define eb emplace_back
mint g = 3;  // 原始根
void fft(vm& a, bool inv = false) {
   int n = si(a), s = my_lg(n);
   static vm z, iz;
   while(si(z) <= s) {
      z.eb(g.pow(mint(-1).x / (1 << si(z))));
      iz.eb(z.back().inv());
   }
   vm b(n);
   repi(i, 1, s + 1) {
      int w = 1 << s - i;
      mint base = inv ? iz[i] : z[i], now = 1;
      for(int y = 0; y < n / 2; y += w) {
         rep(x, w) {
            auto l = a[y << 1 | x], r = now * a[y << 1 | x |
w];
            b[y | x] = l + r, b[y | x | n >> 1] = l - r;
         }
         now *= base;
      }
      swap(a, b);
   }
}

vm mul(vm a, vm b) {
   int n = si(a), m = si(b);
```

```
   if(!n or !m) return {};
   if(min(n, m) <= 30) {
      vm ans(n + m - 1);
      rep(i, n) rep(j, m) ans[i + j] += a[i] * b[j];
      return ans;
   }
   int N = n + m - 1;
   int z = bit_ceil(unsigned(N));
   a.resize(z), b.resize(z);
   fft(a), fft(b);
   rep(i, z) a[i] *= b[i];
   fft(a, true);
   a.resize(n + m - 1);
   mint iz = mint(z).inv();
   fore(e, a) e *= iz;
   return a;
}
```

modint.hpp

md5: 3db9f2

```
constexpr int mod = 998244353;
struct mint {
   int x;
   mint(ll x_ = 0) : x(x_ % mod) {
      if(x < 0) x += mod;
   }
   mint operator-() {
      auto res = *this;
      res.x = (x ? mod - x : 0);
      return res;
   }
   mint& operator+=(mint r) {
      if((x += r.x) >= mod) x -= mod;
      return *this;
   }
   mint& operator-=(mint r) {
      if((x -= r.x) < 0) x += mod;
      return *this;
   }
   mint& operator*=(mint r) {
      x = 1LL * x * r.x % mod;
      return *this;
   }
   mint& operator/=(mint r) { return *this *= r.inv(); }
   friend mint operator+(mint a, mint b) { return a += b; }
   friend mint operator-(mint a, mint b) { return a -= b; }
   friend mint operator*(mint a, mint b) { return a *= b; }
   friend mint operator/(mint a, mint b) { return a /= b; }
   mint inv() const { return pow(mod - 2); }
   mint pow(ll b) const {
      mint a = *this, c = 1;
      while(b) {
         if(b & 1) c *= a;
         a *= a;
         b >>= 1;
      }
      return c;
   }
};
using vm = vector<mint>;
```

extra

modint_fast.hpp

md5: 4b6639

```
const uint32_t mod = 998244353;
struct mm {
   uint32_t x;
   mm() : x(0) {}
   template<class T> mm(T x_) : x(x_ % mod) {
      if(x >= mod) x += mod;
   }
   friend mm operator+(mm a, mm b) {
      a.x += b.x;
      if(a.x >= mod) a.x -= mod;
      return a;
   }
   friend mm operator-(mm a, mm b) {
      a.x -= b.x;
      if(a.x >= mod) a.x += mod;
      return a;
   }
   friend mm operator*(mm a, mm b) {
      return (uint64_t)a.x * b.x;
   }
   friend mm operator/(mm a, mm b) { return a * b.inv(); }
   friend mm& operator+=(mm& a, mm b) { return a = a + b; }
   friend mm& operator-=(mm& a, mm b) { return a = a - b; }
   friend mm& operator*=(mm& a, mm b) { return a = a * b; }
   friend mm& operator/=(mm& a, mm b) {
      return a = a * b.inv();
   }
   mm inv() const { return pow(mod - 2); }
   mm pow(ll b) const {
      mm a = *this, c = 1;
      while(b) {
         if(b & 1) c *= a;
         a *= a;
         b >>= 1;
      }
      return c;
   }
};
```

FPS

FFT.hpp

md5: 63d23d

```
// {998244353, 3}, {1811939329, 13}, {2013265921, 31}
mint g = 3;  // 原始根
void fft(vector<mint>& a) {
   ll n = sz(a), lg = __lg(n);
   assert((1 << lg) == n);
   vector<mint> b(n);
   repi(l, 1, lg + 1) {
      ll w = n >> l;
      mint s = 1, r = g.pow(mod >> l);
      for(ll u = 0; u < n / 2; u += w) {
         repi(d, 0, w) {
            mint x = a[u << 1 | d], y = a[u << 1 | w | d] * s;
            b[u | d] = x + y;
            b[n >> 1 | u | d] = x - y;
         }
         s *= r;
      }
      swap(a, b);
   }
}
vector<mint> conv(vector<mint> a, vector<mint> b) {
   if(a.empty() || b.empty()) return {};
   size_t s = sz(a) + sz(b) - 1, n = bit_ceil(s);
   // if(min(sz(a), sz(b)) <= 60) 愚直に掛け算
   a.resize(n);
   b.resize(n);
   fft(a);
   fft(b);
   mint inv = mint(n).inv();
   repi(i, 0, n) a[i] *= b[i] * inv;
   reverse(1 + all(a));
   fft(a);
   a.resize(s);
   return a;
}
```

FFT_fast.hpp

md5: d932d5

```
// modint を u32 にして加減算を真面目にやると速い
mint g = 3;  // 原始根
void fft(vector<mint>& a) {
   ll n = sz(a), lg = __lg(n);
   static auto z = [] {
      vector<mint> z(30);
      mint s = 1;
      repi(i, 2, 32) {
         z[i - 2] = s * g.pow(mod >> i);
         s *= g.inv().pow(mod >> i);
      }
      return z;
   }();
   repi(l, 0, lg) {
      ll w = 1 << (lg - l - 1);
      mint s = 1;
      repi(k, 0, 1 << l) {
         ll o = k << (lg - l);
         repi(i, o, o + w) {
            mint x = a[i], y = a[i + w] * s;
            a[i] = x + y;
            a[i + w] = x - y;
```

```
        }
        s *= z[countr_zero<uint64_t>(~k)];
      }
   }
}
// コピペ
void ifft(vector<mint>& a) {
   ll n = sz(a), lg = __lg(n);
   static auto z = [] {
      vector<mint> z(30);
      mint s = 1;
      repi(i, 2, 32) {  // g を逆数に
         z[i - 2] = s * g.inv().pow(mod >> i);
         s *= g.pow(mod >> i);
      }
      return z;
   }();
   for(ll l = lg; l--;) {  // 逆順に
      ll w = 1 << (lg - l - 1);
      mint s = 1;
      repi(k, 0, 1 << l) {
         ll o = k << (lg - l);
         repi(i, o, o + w) {
            mint x = a[i], y = a[i + w];  // *s を下に移動
            a[i] = x + y;
            a[i + w] = (x - y) * s;
         }
         s *= z[countr_zero<uint64_t>(~k)];
      }
   }
}
vector<mint> conv(vector<mint> a, vector<mint> b) {
   if(a.empty() || b.empty()) return {};
   size_t s = sz(a) + sz(b) - 1, n = bit_ceil(s);
   // if(min(sz(a), sz(b)) <= 60) 愚直に掛け算
   a.resize(n);
   b.resize(n);
   fft(a);
   fft(b);
   mint inv = mint(n).inv();
   repi(i, 0, n) a[i] *= b[i] * inv;
   ifft(a);
   a.resize(s);
   return a;
}
```

graph

TreeHash.hpp

md5: c07d91

```
using ull = unsigned long long;
struct TreeHash {
   const vvi& g;
   vector<ull> h, rnd;
   vi ht;
   mt19937_64 mt{(ull)chrono::steady_clock::now()
                     .time_since_epoch()
                     .count()};
   TreeHash(const vvi& g) : g(g), h(sz(g)), ht(sz(g)) {}
   ull operator()(vi& ng, int r) {
      int sv = ng[r];
      ng[r] = 1, dfs(ng, r), ng[r] = sv;
      return h[r];
   }
   void dfs(vi& ng, int v) {
      h[v] = 1, ht[v] = 0;
      for(int u : g[v])
         if(!ng[u]) {
            ng[u] = 1, dfs(ng, u), ng[u] = 0;
            while(sz(rnd) <= ht[u]) rnd.push_back(mt());
            h[v] *= (h[u] + rnd[ht[u]]) | 1;
            chmax(ht[v], ht[u] + 1);
         }
   }
};
```

Trie.hpp

md5: 63ded2

```
struct Trie {
   int cz;
   vvi g;
   int size;
   vll dp;
   Trie(int cz = 26) : cz(cz), g(1, vi(cz)), size(1), dp(1) {}
   void add(vi& s) {
      int si = s.size();
      int id = 0;
      rep(j, si) {
         if(g[id][s[j]]) {
            id = g[id][s[j]];
         } else {
            g[id][s[j]] = size;
            id = size;
            size++;
            g.push_back(vi(cz));
            dp.push_back(0);
         }
         dp[id]++;
      }
   }
};
```

centroid_decomposition.hpp

md5: dae9cb

```
template<typename G> struct CentroidDecomposition {
   const G& g;
   vector<int> sub;
   vector<vector<int> > belong;
   vector<bool> v;

   CentroidDecomposition(const G& g)
       : g(g), sub(g.size()), v(g.size()), belong(g.size()) {}

   inline int build_dfs(int idx, int par) {
      sub[idx] = 1;
      for(auto& to : g[idx]) {
         if(to == par || v[to]) continue;
         sub[idx] += build_dfs(to, idx);
      }
      return sub[idx];
   }

   inline int search_centroid(int idx, int par, const int mid)
{
      for(auto& to : g[idx]) {
         if(to == par || v[to]) continue;
         if(sub[to] > mid) return search_centroid(to, idx,
mid);
      }
      return idx;
   }

   inline void belong_dfs(int idx, int par, int centroid) {
      belong[idx].emplace_back(centroid);
      for(auto& to : g[idx]) {
         if(to == par || v[to]) continue;
         belong_dfs(to, idx, centroid);
      }
   }

   inline int build(G& t, int idx) {
      int centroid = search_centroid(idx, -1,
                                     build_dfs(idx, -1) / 2);
      v[centroid] = true;
      belong_dfs(centroid, -1, centroid);
      for(auto& to : g[centroid]) {
         if(!v[to]) t[centroid].emplace_back(build(t, to));
      }
      v[centroid] = false;
      return centroid;
   }

   inline int build(G& t) {
      t.resize(g.size());
      return build(t, 0);
   }
};

vll anss;
void solve(int test) {
   int n;
   cin >> n;
   vvi g(n);
   rep(i, n - 1) {
      int a, b;
      cin >> a >> b;
      a--;
      b--;
      g[a].push_back(b);
```

```
      g[b].push_back(a);
   }
   CentroidDecomposition<vvi> cd(g);
   vvi tr;
   int rt = cd.build(tr);
   rep(i, n) {
      cout << i << " ";
      for(auto u : tr[i]) cout << u << " ";
      cout << endl;
   }
}
```

dijkstra.hpp

md5: e9adaf

```
auto f = [&](int s) {
   vll dist(n, inf);
   priority_queue<P, vector<P>, greater<P>> q;
   dist[s] = 0;
   q.push(P(dist[s], s));
   while(!q.empty()) {
      int v = q.top().second;
      ll d = q.top().first;
      q.pop();
      if(dist[v] < d) continue;
      for(auto u : g[v]) {
         if(chmin(dist[u.to], dist[v] + u.cost)) {
            q.push(P(dist[u.to], u.to));
         }
      }
   }
};
```

edge.hpp

md5: 863cfa

```
struct edge {
   int to;
   ll cost;
   int id;
   edge(int to, ll cost = 1, int id = -1)
       : to(to), cost(cost), id(id) {}
};
using vve = vector<vector<edge>>;
```

eulertrail.hpp

md5: 83e15f

```
// https://kokiymgch.hatenablog.com/entry/2017/12/07/193238
// gが隣接リスト
// gを破壊する
vector<int> EulerianTrail(const int s, vector<vector<int>>& g,
                          const bool directed) {
   function<void(int, vector<int>&)> dfs =
       [&](int u, vector<int>& trail) {
          while(!g[u].empty()) {
             int v = g[u].back();
             g[u].pop_back();
             if(!directed) {
                for(int i = 0; i < g[v].size(); i++) {
                   if(g[v][i] == u) {
                      g[v].erase(g[v].begin() + i);
                      break;
                   }
                }
             }
             dfs(v, trail);
          }
          trail.push_back(u);
       };
   vector<int> trail;
   dfs(s, trail);
   reverse(trail.begin(), trail.end());
   return trail;
}
```

hld.hpp

md5: dbcdf8

```
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/tree/hld.hpp
#define si(a) (int)(size(a))
#define fore(e, v) for(auto&& e : v)
template<typename G> struct HLD {
   int n;
   G& g;
   vector<int> sub, in, out, head, rev, par, d;
   HLD(G& g)
       : n(si(g)),
         g(g),
         sub(n),
         in(n),
         out(n),
         head(n),
         rev(n),
         par(n),
         d(n) {}
   void dfs1(int x, int p) {
      par[x] = p;
      sub[x] = 1;
      if(g[x].size() and g[x][0] == p)
         swap(g[x][0], g[x].back());
      fore(e, g[x]) {
         if(e == p) continue;
         d[e] = d[x] + 1;
         dfs1(e, x);
         sub[x] += sub[e];
         if(sub[g[x][0]] < sub[e]) swap(g[x][0], e);
      }
   }
   void dfs2(int x, int p, int& t) {
      in[x] = t++;
      rev[in[x]] = x;
      fore(e, g[x]) {
         if(e == p) continue;
         head[e] = (g[x][0] == e ? head[x] : e);
         dfs2(e, x, t);
      }
      out[x] = t;
   }
   void build() {
      int t = 0;
      head[0] = 0;
      dfs1(0, -1);
      dfs2(0, -1, t);
   }
   int la(int v, int k) {
      while(1) {
         int u = head[v];
         if(in[v] - k >= in[u]) return rev[in[v] - k];
         k -= in[v] - in[u] + 1;
         v = par[u];
      }
   }
   int lca(int u, int v) {
      for(;; v = par[head[v]]) {
         if(in[u] > in[v]) swap(u, v);
         if(head[u] == head[v]) return u;
      }
   }
   template<typename T, typename Q, typename F>
   T query(int u, int v, const T& e, const Q& q, const F& f,
           bool edge = false) {
      T l = e, r = e;
      for(;; v = par[head[v]]) {
         if(in[u] > in[v]) swap(u, v), swap(l, r);
         if(head[u] == head[v]) break;
         l = f(q(in[head[v]], in[v] + 1), l);
      }
      return f(f(q(in[u] + edge, in[v] + 1), l), r);
   }
   int dist(int u, int v) {
      return d[u] + d[v] - 2 * d[lca(u, v)];
   }
   int jump(int s, int t, int i) {
      if(!i) return s;
      int l = lca(s, t);
      int dst = d[s] + d[t] - d[l] * 2;
      if(dst < i) return -1;
      if(d[s] - d[l] >= i) return la(s, i);
      i -= d[s] - d[l];
      return la(t, d[t] - d[l] - i);
   }
};
// 以下、使い方
using S = ll;
S op(S a, S b) { return a + b; }
S e() { return 0ll; }
void solve(int test) {
   int n;
   cin >> n;
   int m = n - 1;
   vi u(m), v(m), w(m);
   vvi g(n);
```

```
   rep(i, m) {
      cin >> u[i] >> v[i] >> w[i];
      u[i]--, v[i]--;
      g[u[i]].push_back(v[i]);
      g[v[i]].push_back(u[i]);
   }
   HLD hld(g);
   hld.build();
   vi edgetoin(m);
   rep(i, m) {
      // 辺を子のinに対応させる
      int a = u[i], b = v[i];
      if(hld.par[b] == a) swap(a, b);
      edgetoin[i] = hld.in[a];
   }
   segtree<S, op, e> seg(n);
   rep(i, m) { seg.set(edgetoin[i], w[i]); }
   auto query = [&](int l, int r) { return seg.prod(l, r); };
   int q;
   cin >> q;
   rep(qi, q) {
      int t;
      cin >> t;
      if(t == 1) {
         int i, x;
         cin >> i >> x;
         i--;
         w[i] = x;
         seg.set(edgetoin[i], w[i]);
      } else {
         int a, b;
         cin >> a >> b;
         a--;
         b--;
         auto res = hld.query(a, b, e(), query, op, true);
         cout << res << endl;
      }
   }
}
int main() {
   cin.tie(0);
   ios::sync_with_stdio(false);
   int t = 1;  // cin >>t;
   rep(test, t) solve(test);
}
```

lowlink.hpp

md5: 07c344

```
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/graph/lowlink.hpp
#define pii pair<int, int>
#define fore(e, v) for(auto&& e : v)
#define eb emplace_back
#define si(a) (int)(size(a))
template<typename G> struct LL {
   int n;
   const G g;
   vi ord, low, arti;
   vector<pii> bridge;

   LL(G g) : n(si(g)), g(g), ord(si(g), -1), low(si(g), -1) {
      int k = 0;
      rep(i, n) {
         if(ord[i] == -1) k = dfs(i, k, -1);
      }
   }

   int dfs(int x, int k, int p) {
      low[x] = (ord[x] = k++);
      int cnt = 0;
      bool is_arti = false, second = false;
      fore(e, g[x]) {
         if(ord[e] == -1) {
            cnt++;
            k = dfs(e, k, x);
            chmin(low[x], low[e]);
            is_arti |= (p != -1) && (low[e] >= ord[x]);
            if(ord[x] < low[e]) bridge.eb(minmax(x, e));
         } else if(e != p or second) {
            chmin(low[x], ord[e]);
         } else {
            second = true;
         }
      }
      is_arti |= p == -1 && cnt > 1;
      if(is_arti) arti.eb(x);
      return k;
   }
};
```

matrix_tree.hpp

md5: 5ce1b7

```
template<typename T> struct Matrix {
   int h, w;
   vector<vector<T>> d;
   Matrix() {}
   Matrix(int h, int w, T val = 0)
       : h(h), w(w), d(h, vector<T>(w, val)) {}
   Matrix& unit() {
      assert(h == w);
      rep(i, h) d[i][i] = 1;
      return *this;
   }
   const vector<T>& operator[](int i) const { return d[i]; }
   vector<T>& operator[](int i) { return d[i]; }
   Matrix operator*(const Matrix& a) const {
      assert(w == a.h);
      Matrix r(h, a.w);
      rep(i, h) rep(k, w) rep(j, a.w) {
         r[i][j] += d[i][k] * a[k][j];
      }
      return r;
   }
   Matrix pow(long long t) const {
      assert(h == w);
      if(!t) return Matrix(h, h).unit();
      if(t == 1) return *this;
      Matrix r = pow(t >> 1);
      r = r * r;
      if(t & 1) r = r * (*this);
      return r;
   }
   mint det() {
      assert(h == w);
      mint res = 1;
      rep(k, h) {
         for(int i = k; i < h; ++i) {
            if(d[i][k] == 0) continue;
            if(i != k) {
               swap(d[i], d[k]);
               res = -res;
            }
         }
         if(d[k][k] == 0) return 0;
         res *= d[k][k];
         mint inv = mint(1) / d[k][k];
         rep(j, h) d[k][j] *= inv;
         for(int i = k + 1; i < h; ++i) {
            mint c = d[i][k];
            for(int j = k; j < h; ++j) d[i][j] -= d[k][j] * c;
         }
      }
      return res;
   }
};
l[i][j] = deg(i) if(i == j) iとjの辺の本数
          * (-1) if(i != j)

              これの余因子が全域木々の個数(-1)
          * *(0 + 0) と0行0列を除いた行列式をかけたものでok
```

scc.hpp

md5: 9d4785

```
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/t
#define fore(e, v) for(auto&& e : v)
#define eb emplace_back
#define si(a) (int)(size(a))
template<typename G> struct SCC {
   G g;
   vector<vi> rg;
   vi comp, ord, used;
   int num;  // 連結成分の数

   SCC(G g)
       : g(g),
         rg(si(g)),
         comp(si(g), -1),
         ord(si(g)),
         used(si(g)) {
```

```
    rep(i, si(g)) fore(e, g[i]) rg[e].eb(i);
    build();
  };
  int operator[](int k) { return comp[k]; }
  void dfs(int x) {
    if(used[x]) return;
    used[x] = true;
    fore(e, g[x]) if(!used[e]) dfs(e);
    ord.eb(x);
  }
  void rdfs(int x, int cnt) {
    if(comp[x] != -1) return;
    comp[x] = cnt;
    fore(e, rg[x]) if(comp[e] == -1) rdfs(e, cnt);
  }
  void build() {
    rep(i, g.size()) dfs(i);
    reverse(all(ord));
    num = 0;
    fore(i, ord) if(comp[i] == -1) { rdfs(i, num), num++; }
  }
};
```

flow

bipartite-matching.hpp

md5: 845322

```
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/flow/bipartite-
matching.hpp
#define eb emplace_back
#define si(a) (int)(size(a))
#define fore(e, v) for(auto&& e : v)
struct Bimatch {
  vector<vi> g;
  vi d, mc, used, vv;
  Bimatch(int n, int m) : g(n), mc(m, -1), used(n) {}
  void add(int u, int v) { g[u].eb(v); }
  void bfs() {
    d.assign(si(g), -1);
    queue<int> q;
    rep(i, si(g)) {
      if(!used[i]) {
        q.emplace(i);
        d[i] = 0;
      }
    }
    while(!q.empty()) {
      int x = q.front();
      q.pop();
      fore(e, g[x]) {
        int c = mc[e];
        if(c >= 0 and d[c] == -1) {
          d[c] = d[x] + 1;
          q.emplace(c);
        }
      }
    }
  }

  bool dfs(int x) {
    vv[x] = true;
    fore(e, g[x]) {
      int c = mc[e];
      if(c < 0 or (!vv[c] and d[c] == d[x] + 1 and dfs(c)))
{
        mc[e] = x;
        used[x] = true;
        return true;
      }
    }
    return false;
  }
  int match() {
    int ret = 0;
    while(true) {
      bfs();
      vv.assign(si(g), false);
      int f = 0;
      rep(i, si(g)) if(!used[i] and dfs(i)) f++;
      if(!f) return ret;
      ret += f;
    }
  }
};
```

maxflow.hpp

md5: 0459fb

```
// Dinic法 O(V^2E)
struct mf_graph {
  struct edge {
    int to;
    ll cap;
    int rev;
  };
  ll V;
  vector<vector<edge>> g;
  vll dis, nxt;
  mf_graph(ll v) : V(v), g(v) {}
  // 返り値は辺のindex
  // あとで g[a][id].cap を変更できる
  ll add_edge(ll a, ll b, ll cap) {
    g[a].push_back({(int)b, cap, (int)sz(g[b])});
    g[b].push_back({(int)a, 0, (int)sz(g[a]) - 1});
    return sz(g[a]) - 1;
  }
  void bfs(ll s) {
    dis.assign(V, -1), dis[s] = 0;
    queue<ll> q;
    q.push(s);
    while(sz(q)) {
      ll v = q.front();
      q.pop();
      for(auto& e : g[v])
        if(e.cap > 0 && dis[e.to] < 0)
          dis[e.to] = dis[v] + 1, q.push(e.to);
    }
  }
  ll dfs(ll v, ll t, ll f) {
    if(v == t) return f;
    for(ll& i = nxt[v]; i < sz(g[v]); i++) {
      auto& e = g[v][i];
      if(e.cap > 0 && dis[v] < dis[e.to]) {
        ll d = dfs(e.to, t, min(f, e.cap));
        if(d > 0) {
          e.cap -= d, g[e.to][e.rev].cap += d;
          return d;
        }
      }
    }
    return 0;
  }
  ll max_flow(ll s, ll t) {
    ll res = 0;
    while(1) {
      bfs(s);
      if(dis[t] < 0) return res;
      nxt.assign(V, 0);
      ll f;
      while((f = dfs(s, t, inf)) > 0) res += f;
    }
  }
};
```

mincostflow.hpp

md5: eddc72

```
// 最小費用流 O(FElogV)。コストは非負
struct mcf_graph {
  struct edge {
    int to;
    ll cap, cost;
    int rev;
  };
  ll V;
  vector<vector<edge>> g;
  vll h, dis, pv, pe;
  mcf_graph(ll v) : V(v), g(v), dis(v), pv(v), pe(v) {}
  void add_edge(ll a, ll b, ll cap, ll cost) {
    g[a].push_back({(int)b, cap, cost, (int)sz(g[b])});
    g[b].push_back({(int)a, 0, -cost, (int)sz(g[a]) - 1});
  }
  ll min_cost_flow(ll s, ll t, ll f) {
    ll res = 0;
    h.assign(V, 0);
    while(f > 0) {
      priority_queue<P, vp, greater<P>> q;
      dis.assign(V, inf), dis[s] = 0;
      q.push({0, s});
```

```
        while(sz(q)) {
           auto [d, v] = q.top();
           q.pop();
           if(dis[v] < d) continue;
           rep(i, sz(g[v])) {
              auto& e = g[v][i];
              ll nd = dis[v] + e.cost + h[v] - h[e.to];
              if(e.cap > 0 && dis[e.to] > nd)
                 dis[e.to] = nd, pv[e.to] = v, pe[e.to] = i,
                 q.push({nd, e.to});
           }
        }
        if(dis[t] == inf) return -1;
        rep(i, V) h[i] += dis[i];
        ll d = f;
        for(ll v = t; v != s; v = pv[v])
           chmin(d, g[pv[v]][pe[v]].cap);
        f -= d, res += d * h[t];
        for(ll v = t; v != s; v = pv[v]) {
           auto& e = g[pv[v]][pe[v]];
           e.cap -= d, g[v][e.rev].cap += d;
        }
     }
     return res;
  }
};
```

燃やす埋める.md

変形前の制約	変形後の制約
x が 0 のとき z 失う	(x, T, z)
x が 0 のとき z 得る	無条件で z 得る; (S, x, z)
x が 1 のとき z 失う	(S, x, z)
x が 1 のとき z 得る	無条件で z 得る; (x, T, z)
$x, y, \ldots$ がすべて 0 のとき z 得る	無条件で z 得る; $(S, w, z), (w, x, \infty), (w, y, \infty)$
$x, y, \ldots$ がすべて 1 のとき z 得る	無条件で z 得る; $(w, T, z), (x, w, \infty), (y, w, \infty)$

string

KMP.hpp

md5: 8f3c46

```
// kmp[i] := max{ l ≤ i | s[:l] == s[(i+1)-l:i+1] }
// abacaba -> 0010123
auto KMP(string s) {
   vector<ll> p(sz(s));
   repi(i, 1, sz(s)) {
      ll g = p[i - 1];
      while(g && s[i] != s[g]) g = p[g - 1];
      p[i] = g + (s[i] == s[g]);
   }
   return p;
}
```

Manacher.hpp

md5: 5882fb

```
// 各位置での回文半径を求める
// aaabaaa -> 1214121
// 偶数長の回文を含めて直径を知るには,
// N+1 個の $ を挿入して 1 を引く
// $a$a$a$b$a$a$a$ -> 123432181234321
auto manacher(string s) {
   ll n = sz(s), i = 0, j = 0;
   vector<ll> r(n);
   while(i < n) {
      while(i >= j && i + j < n && s[i - j] == s[i + j]) j++;
      r[i] = j;
      ll k = 1;
      while(i >= k && i + k < n && k + r[i - k] < j) {
         r[i + k] = r[i - k];
         k++;
      }
      i += k, j -= k;
   }
   return r;
}
```

lcp_array.hpp

md5: 2c8f78

```
// https://github.com/YNUCPC/ynu-icpc-
library/blob/main/src/string/lcp_array.hpp
//  lcp[i] = sa[i]とsa[i+1]の common prefix の長さ
vector<int> lcp_array(const string& s, const vector<int>& sa) {
   int n = sa.size();
   vector<int> lcp(n), rank(n);  // rank[i] = s[i,n)の辞書順位
   for(int i = 0; i < n; i++) rank[sa[i]] = i;
   for(int i = 0, len = 0; i < n; i++) {
      if(len > 0) len--;
      if(rank[i] == n - 1) continue;
      int j = sa
          [rank[i]
           + 1];  // s[i,n)より辞書順で1つ大きいsuffixの先頭位置
      while(max(i, j) + len < int(s.size())
            && s[i + len] == s[j + len])
         len++;
      lcp[rank[i]] = len;
   }
   return lcp;
}

/*
string s; cin >> s;
int n=s.size();
auto sa=suffix_array(s);
auto lcp=lcp_array(s,sa);
vi id(n+1);
rep(i,sa.size())id[sa[i]]=i;
segtree<S,op,e> seg(lcp);
vvm dp(n,vm(n));
repi(l,1,n)repi(r,l+1,n){
int len=seg.prod(min(id[l],id[r]),max(id[l],id[r]));
*/
```

palindrome.hpp

md5: c3ad5a

```
vi manachar(string S) {
   int i = 0, j = 0;
   vi R(S.size());
   while(i < S.size()) {
      while(i - j >= 0 && i + j < S.size()
            && S[i - j] == S[i + j])
         ++j;
      R[i] = j;
      int k = 1;
      while(i - k >= 0 && k + R[i - k] < j)
         R[i + k] = R[i - k], ++k;
      i += k;
      j -= k;
   }
   return R;
}
// 奇数,偶数の最大半径
pair<vi, vi> kaibun(string s) {
   vi l = manachar(s);
   string t = "";
   int n = s.size();
   rep(i, n) {
      t += s[i];
      if(i != n - 1) t += '@';
   }
   vi v = manachar(t);
   vi r;
   for(int i = 1; i < v.size(); i += 2) {
      if(i >= v.size()) break;
      r.push_back(v[i] / 2);
   }
   return make_pair(l, r);
}
```

rolling_hash.hpp

md5: a0c5d3

```
// mod 2^61-1、基数はランダム
// b と p は全インスタンスで共有されるので
// 別の文字列同士も比較できる
using ull = unsigned long long;
struct RH {
   static constexpr ull M = (1ull << 61) - 1;
   static inline vector<ull> p = {1};
   static inline ull b = 0;
   vector<ull> h;
   static ull mul(ull x, ull y) {
```

```
      __uint128_t z = (__uint128_t)x * y;
      ull w = (ull)(z >> 61) + (ull)(z & M);
      return w >= M ? w - M : w;
   }
   static void ext(ll n) {
      static mt19937_64 r(random_device{}());
      if(!b) b = M / 2 + r() % (M / 2);
      while(sz(p) <= n) p.push_back(mul(p.back(), b));
   }
   // x の後ろに、長さ ly のハッシュ y をつなぐ
   static ull concat(ull x, ull y, ll ly) {
      ext(ly);
      ull w = mul(x, p[ly]) + y;
      return w >= M ? w - M : w;
   }
   RH() : h{0} {}
   // string でも vi/vll でもよい
   // 値は 0 以上 2^61-1 未満
   // 負が混じるなら全体に下駄を履かせる
   // 0 を含む数列で長さの違う区間どうしを比べるなら
   // 全体に +1 しておくと安全
   template<class T> RH(const T& s) : h{0} {
      ext(sz(s));
      for(auto c : s) {
         ull w = mul(h.back(), b) + c;
         h.push_back(w >= M ? w - M : w);
      }
   }
   ull get(ll l, ll r) {  // s[l,r)
      ull w = h[r] + M - mul(h[l], p[r - l]);
      return w >= M ? w - M : w;
   }
};
```

suffix_array.hpp

md5: 79d0c6

```
// lcp先頭変なのはいっていそう
//  returns pair{sa, lcp}
//  sa 長さ n : s[sa[0]:] < s[sa[1]:] < … < s[sa[n-1]:]
//  lcp 長さ n-1 : lcp[i] = LCP(s[sa[i]:], s[sa[i+1]:])
#define si(a) (int)(size(a))
auto SA(string s) {
   ll n = si(s) + 1, lim = 256;
   // assert(lim > ranges::max(s));
   vector<ll> sa(n), lcp(n), x(all(s) + 1), y(n),
       ws(max(n, lim)), rk(n);
   iota(all(sa), 0);
   for(ll j = 0, p = 0; p < n; j = max(1LL, j * 2), lim = p) {
      p = j;
      iota(all(y), n - j);
      repi(i, 0, n) if(sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      repi(i, 0, n) ws[x[i]]++;
      repi(i, 1, lim) ws[i] += ws[i - 1];
      for(ll i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y);
      p = 1;
      x[sa[0]] = 0;
      repi(i, 1, n) {
         ll a = sa[i - 1], b = sa[i];
         x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1
                                                      : p++;
      }
   }
   repi(i, 1, n) rk[sa[i]] = i;
   for(ll i = 0, k = 0; i < n - 1; lcp[rk[i++]] = k) {
      if(k) k--;
      while(s[i + k] == s[sa[rk[i] - 1] + k]) k++;
   }
   sa.erase(begin(sa));
   lcp.erase(begin(lcp));
   return pair{sa, lcp};
}
```

z_algorithm.hpp

md5: 146991

```
// https://github.com/YNUCPC/ynu-icpc-
library/blob/main/src/string/z_algorithm.hpp
vector<int> z_algorithm(string& s) {
   int n = int(s.size());
   vector<int> z(n);
   z[0] = n;
   for(int i = 1, l = 0, r = 0; i < n; i++) {
      int& k = z[i];
      k = (r <= i ? 0 : min(r - i, z[i - l]));
      while(i + k < n && s[k] == s[i + k]) k++;
      if(r < i + k) l = i, r = i + k;
   }
   return z;
}
```

geometry

memo

Bi-Connected-Components.hpp

md5: eddd4d

```
template<typename G> struct BiConnectedComponents : LowLink<G>
{
   using LL = LowLink<G>;

   vector<int> used;
   vector<vector<pair<int, int> > > bc;
   vector<pair<int, int> > tmp;

   BiConnectedComponents(const G& g) : LL(g) {}

   void dfs(int idx, int par) {
      used[idx] = true;
      for(auto& to : this->g[idx]) {
         if(to == par) continue;
         if(!used[to] || this->ord[to] < this->ord[idx]) {
            tmp.emplace_back(minmax(idx, to));
         }
         if(!used[to]) {
            dfs(to, idx);
            if(this->low[to] >= this->ord[idx]) {
               bc.emplace_back();
               for(;;) {
                  auto e = tmp.back();
                  bc.back().emplace_back(e);
                  tmp.pop_back();
                  if(e.first == min(idx, to)
                     && e.second == max(idx, to)) {
                     break;
                  }
               }
            }
         }
      }
   }

   void build() override {
      LL::build();
      used.assign(this->g.size(), 0);
      for(int i = 0; i < used.size(); i++) {
         if(!used[i]) dfs(i, -1);
      }
   }
};
```

使用例

```
template<int mod> struct ModInt {
   int x;

   ModInt() : x(0) {}

   ModInt(int64_t y)
       : x(y >= 0 ? y % mod : (mod - (-y) % mod) % mod) {}

   ModInt& operator+=(const ModInt& p) {
      if((x += p.x) >= mod) x -= mod;
      return *this;
   }

   ModInt& operator-=(const ModInt& p) {
      if((x += mod - p.x) >= mod) x -= mod;
      return *this;
   }

   ModInt& operator*=(const ModInt& p) {
      x = (int)(1LL * x * p.x % mod);
      return *this;
   }

   ModInt& operator/=(const ModInt& p) {
      *this *= p.inverse();
      return *this;
   }
```

```
   ModInt operator-() const { return ModInt(-x); }

   ModInt operator+(const ModInt& p) const {
      return ModInt(*this) += p;
   }

   ModInt operator-(const ModInt& p) const {
      return ModInt(*this) -= p;
   }

   ModInt operator*(const ModInt& p) const {
      return ModInt(*this) *= p;
   }

   ModInt operator/(const ModInt& p) const {
      return ModInt(*this) /= p;
   }

   bool operator==(const ModInt& p) const { return x == p.x; }

   bool operator!=(const ModInt& p) const { return x != p.x; }

   ModInt inverse() const {
      int a = x, b = mod, u = 1, v = 0, t;
      while(b > 0) {
         t = a / b;
         swap(a -= t * b, b);
         swap(u -= t * v, v);
      }
      return ModInt(u);
   }

   ModInt pow(int64_t n) const {
      ModInt ret(1), mul(x);
      while(n > 0) {
         if(n & 1) ret *= mul;
         mul *= mul;
         n >>= 1;
      }
      return ret;
   }

   friend ostream& operator<<(ostream& os, const ModInt& p) {
      return os << p.x;
   }

   friend istream& operator>>(istream& is, ModInt& a) {
      int64_t t;
      is >> t;
      a = ModInt<mod>(t);
      return (is);
   }
};

using modint = ModInt<mod>;

template<typename T> struct Combination {
   vector<T> _fact, _rfact;

   Combination(int sz) : _fact(sz + 1), _rfact(sz + 1) {
      _fact[0] = _rfact[sz] = 1;
      for(int i = 1; i <= sz; i++) _fact[i] = _fact[i - 1] * i;
      _rfact[sz] /= _fact[sz];
      for(int i = sz - 1; i >= 0; i--)
         _rfact[i] = _rfact[i + 1] * (i + 1);
   }

   inline T fact(int k) const { return _fact[k]; }

   inline T rfact(int k) const { return _rfact[k]; }

   T P(int n, int r) const {
      if(r < 0 || n < r) return 0;
      return fact(n) * rfact(n - r);
   }

   T C(int p, int q) const {
      if(q < 0 || p < q) return 0;
      return fact(p) * rfact(q) * rfact(p - q);
   }

   T H(int n, int r) const {
      if(n < 0 || r < 0) return (0);
      return r == 0 ? 1 : C(n + r - 1, r);
   }
};
```

```
int64_t euler_phi(int64_t n) {
   int64_t ret = n;
   for(int64_t i = 2; i * i <= n; i++) {
      if(n % i == 0) {
         ret -= ret / i;
         while(n % i == 0) n /= i;
      }
   }
   if(n > 1) ret -= ret / n;
   return ret;
}

int main() {
   int N, M, K;
   cin >> N >> M >> K;
   UnWeightedGraph g(N);
   Combination<modint> beet(101010);
   while(M--) {
      int x, y;
      cin >> x >> y;
      --x, --y;
      g[x].push_back(y);
      g[y].push_back(x);
   }
   BiConnectedComponents<UnWeightedGraph> bcc(g);
   bcc.build();
   modint ret = 1;
   for(auto& vs : bcc.bc) {
      set<int> vertex;
      for(auto& p : vs) vertex.emplace(p.first);
      for(auto& p : vs) vertex.emplace(p.second);
      if(vertex.size() == vs.size() + 1) {
         ret *= K;
      } else if(vertex.size() == vs.size()) {
         modint add = 0;
         for(int i = 1; i <= vertex.size(); i++) {
            if(vertex.size() % i == 0) {
               add += modint(K).pow(vertex.size() / i)
                      * euler_phi(i);
            }
         }
         add /= vertex.size();
         ret *= add;
      } else {
         ret *= beet.H(K, vs.size());
      }
   }
   cout << ret << endl;
}
```

Primes.md

素数の個数

n	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9	10^{10}
$\pi(n)$	25	168	1229	9592	78498	664579	5.76e+6	5.08e+7	4.55e+8

高度合成数

$\leq n$	10^3	10^4	10^5	10^6	10^7	10^8	10^9
x	840	7560	83160	720720	8648640	73513440	735134400
$d^0(x)$	32	64	128	240	448	768	1344

$\leq n$	10^{10}	10^{11}	10^{12}	10^{13}	10^{14}	10^{15}	10^{16}	10^{17}	10^{18}
$d^0(x)$	2304	4032	6720	10752	17280	26880	41472	64512	103680

素数階乗

n	2	3	5	7	11	13	17	19	23	29
$n\#$	2	6	30	210	2310	30030	510510	9.70e+6	2.23e+8	6.47e+9

階乗

4!	5!	6!	7!	8!	9!	10!	11!	12!	13!
24	120	720	5040	40320	362880	3.63e+6	3.99e+7	4.79e+8	6.23e+9

dynamic_segtree.hpp

md5: 279c02

```
template<typename T> class dynamic_segtree {
   private:
   struct node {
```

```
    T val, lazy;
    int st_size;
    node *par, *left, *right;
    node()
        : val(id2),
          lazy(id1),
          st_size(0),
          par(nullptr),
          left(nullptr),
          right(nullptr) {}
};
void opr1(T& arg1, T arg2) { arg1 += arg2; }
T opr2(T arg1, T arg2) { return arg1 + arg2; }
int nw_size;
static const int POOL_SIZE = 4000000;
static const int MAX_SIZE = 100001;
static node* root;
static node pool[POOL_SIZE];
static const T id1 = (T)0;
static const T id2 = (T)0;
node* alloc() {
   assert(nw_size < POOL_SIZE);
   return (&pool[nw_size++]);
}
void eval(node* k, int l, int r) {
   if(k->lazy != id1) {
      opr1(k->val, k->lazy);
      if(r - l > 1) {
         if(k->left) {
            opr1(k->left->lazy,
                 k->lazy / k->st_size * k->left->st_size);
         }
         if(k->right) {
            opr1(k->right->lazy,
                 k->lazy / k->st_size * k->right->st_size);
         }
      }
      k->lazy = id1;
   }
}

public:
dynamic_segtree() : nw_size(0) { root = alloc(); }
// a 番目に x を追加
// (すでに a 番目に値が存在すればそれに x を足す)
void insert(int a, T x, node* k = root, int l = 0,
            int r = MAX_SIZE) {
   eval(k, l, r);
   k->st_size++;
   k->val = opr2(k->val, x);
   if(r - l == 1) { return; }
   if(a < (l + r) / 2) {
      if(!(k->left)) {
         k->left = alloc();
         k->left->par = k;
      }
      insert(a, x, k->left, l, (l + r) / 2);
   } else {
      if(!(k->right)) {
         k->right = alloc();
         k->right->par = k;
      }
      insert(a, x, k->right, (l + r) / 2, r);
   }
}
void range(int a, int b, const T x, node* k = root,
           int l = 0, int r = MAX_SIZE) {
   eval(k, l, r);
   if(b <= l || r <= a) return;
   if(a <= l && r <= b) {
      opr1(k->lazy, k->st_size * x);
      eval(k, l, r);
      return;
   }
   k->val = id2;
   if(k->left) {
      range(a, b, x, k->left, l, (l + r) / 2);
      k->val = opr2(k->val, k->left->val);
   }
   if(k->right) {
      range(a, b, x, k->right, (l + r) / 2, r);
      k->val = opr2(k->val, k->right->val);
   }
}
T query(int a, int b, node* k = root, int l = 0,
        int r = MAX_SIZE) {
   eval(k, l, r);
   if(b <= l || r <= a) { return id2; }
   if(a <= l && r <= b) { return k->val; }
   T vl = id2, vr = id2;
   if(k->left) { vl = query(a, b, k->left, l, (l + r) / 2);
}
   if(k->right) {
      vr = query(a, b, k->right, (l + r) / 2, r);
   }
   return opr2(vl, vr);
}
};
template<typename T>
class dynamic_segtree<T>::node* dynamic_segtree<T>::root;
template<typename T> class dynamic_segtree<T>::node
    dynamic_segtree<T>::pool[POOL_SIZE];
```

geometry.hpp

md5: 49dca1

```
using Real = double;
using Point = complex<Real>;
const Real EPS = 1e-8, PI = acos(-1);

inline bool eq(Real a, Real b) { return fabs(b - a) < EPS; }

Point operator*(const Point& p, const Real& d) {
   return Point(real(p) * d, imag(p) * d);
}

istream& operator>>(istream& is, Point& p) {
   Real a, b;
   is >> a >> b;
   p = Point(a, b);
   return is;
}

ostream& operator<<(ostream& os, Point& p) {
   return os << fixed << setprecision(10) << p.real() << " "
             << p.imag();
}

// 点 p を反時計回りに theta 回転
Point rotate(Real theta, const Point& p) {
   return Point(cos(theta) * p.real() - sin(theta) * p.imag(),
                sin(theta) * p.real() + cos(theta) * p.imag());
}

Real radian_to_degree(Real r) { return (r * 180.0 / PI); }

Real degree_to_radian(Real d) { return (d * PI / 180.0); }

// a-b-c の角度のうち小さい方を返す
Real get_angle(const Point& a, const Point& b, const Point& c)
{
   const Point v(b - a), w(c - b);
   Real alpha = atan2(v.imag(), v.real()),
        beta = atan2(w.imag(), w.real());
   if(alpha > beta) swap(alpha, beta);
   Real theta = (beta - alpha);
   return min(theta, 2 * acos(-1) - theta);
}

namespace std {
bool operator<(const Point& a, const Point& b) {
   return a.real() != b.real() ? a.real() < b.real()
                               : a.imag() < b.imag();
}
}  // namespace std

struct Line {
   Point a, b;

   Line() = default;

   Line(Point a, Point b) : a(a), b(b) {}

   Line(Real A, Real B, Real C)  // Ax + By = C
   {
      if(eq(A, 0)) a = Point(0, C / B), b = Point(1, C / B);
      else if(eq(B, 0))
         b = Point(C / A, 0), b = Point(C / A, 1);
      else a = Point(0, C / B), b = Point(C / A, 0);
   }

   friend ostream& operator<<(ostream& os, Line& p) {
```

```
      return os << p.a << " to " << p.b;
   }

   friend istream& operator>>(istream& is, Line& a) {
      return is >> a.a >> a.b;
   }
};

struct Segment : Line {
   Segment() = default;

   Segment(Point a, Point b) : Line(a, b) {}
};

struct Circle {
   Point p;
   Real r;

   Circle() = default;

   Circle(Point p, Real r) : p(p), r(r) {}
};

using Points = vector<Point>;
using Polygon = vector<Point>;
using Segments = vector<Segment>;
using Lines = vector<Line>;
using Circles = vector<Circle>;

Real cross(const Point& a, const Point& b) {
   return real(a) * imag(b) - imag(a) * real(b);
}

Real dot(const Point& a, const Point& b) {
   return real(a) * real(b) + imag(a) * imag(b);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_1_C
// 点の回転方向
int ccw(const Point& a, Point b, Point c) {
   b = b - a, c = c - a;
   if(cross(b, c) > EPS) return +1;   // "COUNTER_CLOCKWISE"
   if(cross(b, c) < -EPS) return -1;  // "CLOCKWISE"
   if(dot(b, c) < 0) return +2;       // "ONLINE_BACK"
   if(norm(b) < norm(c)) return -2;   // "ONLINE_FRONT"
   return 0;                          // "ON_SEGMENT"
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_A
// 平行判定
bool parallel(const Line& a, const Line& b) {
   return eq(cross(a.b - a.a, b.b - b.a), 0.0);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_A
// 垂直判定
bool orthogonal(const Line& a, const Line& b) {
   return eq(dot(a.a - a.b, b.a - b.b), 0.0);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_1_A
// 射影
// 直線 l に p から垂線を引いた交点を求める
Point projection(const Line& l, const Point& p) {
   double t = dot(p - l.a, l.a - l.b) / norm(l.a - l.b);
   return l.a + (l.a - l.b) * t;
}

Point projection(const Segment& l, const Point& p) {
   double t = dot(p - l.a, l.a - l.b) / norm(l.a - l.b);
   return l.a + (l.a - l.b) * t;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_1_B
// 反射
// 直線 l を対称軸として点 p  と線対称にある点を求める
Point reflection(const Line& l, const Point& p) {
   return p + (projection(l, p) - p) * 2.0;
}

bool intersect(const Line& l, const Point& p) {
   return abs(ccw(l.a, l.b, p)) != 1;
}

bool intersect(const Line& l, const Line& m) {
   return abs(cross(l.b - l.a, m.b - m.a)) > EPS
          || abs(cross(l.b - l.a, m.b - l.a)) < EPS;
}

bool intersect(const Segment& s, const Point& p) {
   return ccw(s.a, s.b, p) == 0;
}

bool intersect(const Line& l, const Segment& s) {
   return cross(l.b - l.a, s.a - l.a)
               * cross(l.b - l.a, s.b - l.a)
          < EPS;
}

Real distance(const Line& l, const Point& p);

bool intersect(const Circle& c, const Line& l) {
   return distance(l, c.p) <= c.r + EPS;
}

bool intersect(const Circle& c, const Point& p) {
   return abs(abs(p - c.p) - c.r) < EPS;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_B
bool intersect(const Segment& s, const Segment& t) {
   return ccw(s.a, s.b, t.a) * ccw(s.a, s.b, t.b) <= 0
          && ccw(t.a, t.b, s.a) * ccw(t.a, t.b, s.b) <= 0;
}

int intersect(const Circle& c, const Segment& l) {
   if(norm(projection(l, c.p) - c.p) - c.r * c.r > EPS)
      return 0;
   auto d1 = abs(c.p - l.a), d2 = abs(c.p - l.b);
   if(d1 < c.r + EPS && d2 < c.r + EPS) return 0;
   if(d1 < c.r - EPS && d2 > c.r + EPS
      || d1 > c.r + EPS && d2 < c.r - EPS)
      return 1;
   const Point h = projection(l, c.p);
   if(dot(l.a - h, l.b - h) < 0) return 2;
   return 0;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_A&lang=jp
int intersect(Circle c1, Circle c2) {
   if(c1.r < c2.r) swap(c1, c2);
   Real d = abs(c1.p - c2.p);
   if(c1.r + c2.r < d) return 4;
   if(eq(c1.r + c2.r, d)) return 3;
   if(c1.r - c2.r < d) return 2;
   if(eq(c1.r - c2.r, d)) return 1;
   return 0;
}

Real distance(const Point& a, const Point& b) {
   return abs(a - b);
}

Real distance(const Line& l, const Point& p) {
   return abs(p - projection(l, p));
}

Real distance(const Line& l, const Line& m) {
   return intersect(l, m) ? 0 : distance(l, m.a);
}

Real distance(const Segment& s, const Point& p) {
   Point r = projection(s, p);
   if(intersect(s, r)) return abs(r - p);
   return min(abs(s.a - p), abs(s.b - p));
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_D
Real distance(const Segment& a, const Segment& b) {
   if(intersect(a, b)) return 0;
   return min({distance(a, b.a), distance(a, b.b),
               distance(b, a.a), distance(b, a.b)});
}
```

```
Real distance(const Line& l, const Segment& s) {
   if(intersect(l, s)) return 0;
   return min(distance(l, s.a), distance(l, s.b));
}

Point crosspoint(const Line& l, const Line& m) {
   Real A = cross(l.b - l.a, m.b - m.a);
   Real B = cross(l.b - l.a, l.b - m.a);
   if(eq(abs(A), 0.0) && eq(abs(B), 0.0)) return m.a;
   return m.a + (m.b - m.a) * B / A;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_C
Point crosspoint(const Segment& l, const Segment& m) {
   return crosspoint(Line(l), Line(m));
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_D
pair<Point, Point> crosspoint(const Circle& c, const Line l) {
   Point pr = projection(l, c.p);
   Point e = (l.b - l.a) / abs(l.b - l.a);
   if(eq(distance(l, c.p), c.r)) return {pr, pr};
   double base = sqrt(c.r * c.r - norm(pr - c.p));
   return {pr - e * base, pr + e * base};
}

pair<Point, Point> crosspoint(const Circle& c,
                              const Segment& l) {
   Line aa = Line(l.a, l.b);
   if(intersect(c, l) == 2) return crosspoint(c, aa);
   auto ret = crosspoint(c, aa);
   if(dot(l.a - ret.first, l.b - ret.first) < 0)
      ret.second = ret.first;
   else ret.first = ret.second;
   return ret;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_E
pair<Point, Point> crosspoint(const Circle& c1,
                              const Circle& c2) {
   Real d = abs(c1.p - c2.p);
   Real a = acos((c1.r * c1.r + d * d - c2.r * c2.r)
                 / (2 * c1.r * d));
   Real t = atan2(c2.p.imag() - c1.p.imag(),
                  c2.p.real() - c1.p.real());
   Point p1 = c1.p
              + Point(cos(t + a) * c1.r, sin(t + a) * c1.r);
   Point p2 = c1.p
              + Point(cos(t - a) * c1.r, sin(t - a) * c1.r);
   return {p1, p2};
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_F
// 点 p を通る円 c の接線
pair<Point, Point> tangent(const Circle& c1, const Point& p2) {
   return crosspoint(c1, Circle(p2, sqrt(norm(c1.p - p2)
                                         - c1.r * c1.r)));
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_G
// 円 c1, c2 の共通接線
Lines tangent(Circle c1, Circle c2) {
   Lines ret;
   if(c1.r < c2.r) swap(c1, c2);
   Real g = norm(c1.p - c2.p);
   if(eq(g, 0)) return ret;
   Point u = (c2.p - c1.p) / sqrt(g);
   Point v = rotate(PI * 0.5, u);
   for(int s : {-1, 1}) {
      Real h = (c1.r + s * c2.r) / sqrt(g);
      if(eq(1 - h * h, 0)) {
         ret.emplace_back(c1.p + u * c1.r,
                          c1.p + (u + v) * c1.r);
      } else if(1 - h * h > 0) {
         Point uu = u * h, vv = v * sqrt(1 - h * h);
         ret.emplace_back(c1.p + (uu + vv) * c1.r,
                          c2.p - (uu + vv) * c2.r * s);
         ret.emplace_back(c1.p + (uu - vv) * c1.r,
                          c2.p - (uu - vv) * c2.r * s);
      }
   }
   return ret;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_3_B
// 凸性判定
bool is_convex(const Polygon& p) {
   int n = (int)p.size();
   for(int i = 0; i < n; i++) {
      if(ccw(p[(i + n - 1) % n], p[i], p[(i + 1) % n]) == -1)
         return false;
   }
   return true;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_4_A
// 凸包
Polygon convex_hull(Polygon& p) {
   int n = (int)p.size(), k = 0;
   if(n <= 2) return p;
   sort(p.begin(), p.end());
   vector<Point> ch(2 * n);
   for(int i = 0; i < n; ch[k++] = p[i++]) {
      while(k >= 2
            && cross(ch[k - 1] - ch[k - 2], p[i] - ch[k - 1])
                  < EPS)
         --k;
   }
   for(int i = n - 2, t = k + 1; i >= 0; ch[k++] = p[i--]) {
      while(k >= t
            && cross(ch[k - 1] - ch[k - 2], p[i] - ch[k - 1])
                  < EPS)
         --k;
   }
   ch.resize(k - 1);
   return ch;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_3_C
// 多角形と点の包含判定
enum { OUT, ON, IN };

int contains(const Polygon& Q, const Point& p) {
   bool in = false;
   for(int i = 0; i < Q.size(); i++) {
      Point a = Q[i] - p, b = Q[(i + 1) % Q.size()] - p;
      if(a.imag() > b.imag()) swap(a, b);
      if(a.imag() <= 0 && 0 < b.imag() && cross(a, b) < 0)
         in = !in;
      if(cross(a, b) == 0 && dot(a, b) <= 0) return ON;
   }
   return in ? IN : OUT;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=1033
// 線分の重複除去
void merge_segments(vector<Segment>& segs) {
   auto merge_if_able = [](Segment& s1, const Segment& s2) {
      if(abs(cross(s1.b - s1.a, s2.b - s2.a)) > EPS)
         return false;
      if(ccw(s1.a, s2.a, s1.b) == 1
         || ccw(s1.a, s2.a, s1.b) == -1)
         return false;
      if(ccw(s1.a, s1.b, s2.a) == -2
         || ccw(s2.a, s2.b, s1.a) == -2)
         return false;
      s1 = Segment(min(s1.a, s2.a), max(s1.b, s2.b));
      return true;
   };

   for(int i = 0; i < segs.size(); i++) {
      if(segs[i].b < segs[i].a) swap(segs[i].a, segs[i].b);
   }
   for(int i = 0; i < segs.size(); i++) {
      for(int j = i + 1; j < segs.size(); j++) {
         if(merge_if_able(segs[i], segs[j])) {
            segs[j--] = segs.back(), segs.pop_back();
         }
      }
   }
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
```

```
id=1033
// 線分アレンジメント
// 任意の2線分の交点を頂点としたグラフを構築する
vector<vector<int> > segment_arrangement(vector<Segment>& segs,
                                         vector<Point>& ps) {
   vector<vector<int> > g;
   int N = (int)segs.size();
   for(int i = 0; i < N; i++) {
      ps.emplace_back(segs[i].a);
      ps.emplace_back(segs[i].b);
      for(int j = i + 1; j < N; j++) {
         const Point p1 = segs[i].b - segs[i].a;
         const Point p2 = segs[j].b - segs[j].a;
         if(cross(p1, p2) == 0) continue;
         if(intersect(segs[i], segs[j])) {
            ps.emplace_back(crosspoint(segs[i], segs[j]));
         }
      }
   }
   sort(begin(ps), end(ps));
   ps.erase(unique(begin(ps), end(ps)), end(ps));

   int M = (int)ps.size();
   g.resize(M);
   for(int i = 0; i < N; i++) {
      vector<int> vec;
      for(int j = 0; j < M; j++) {
         if(intersect(segs[i], ps[j])) { vec.emplace_back(j); }
      }
      for(int j = 1; j < vec.size(); j++) {
         g[vec[j - 1]].push_back(vec[j]);
         g[vec[j]].push_back(vec[j - 1]);
      }
   }
   return (g);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_4_C
// 凸多角形の切断
// 直線 l.a-l.b で切断しその左側にできる凸多角形を返す
Polygon convex_cut(const Polygon& U, Line l) {
   Polygon ret;
   for(int i = 0; i < U.size(); i++) {
      Point now = U[i], nxt = U[(i + 1) % U.size()];
      if(ccw(l.a, l.b, now) != -1) ret.push_back(now);
      if(ccw(l.a, l.b, now) * ccw(l.a, l.b, nxt) < 0) {
         ret.push_back(crosspoint(Line(now, nxt), l));
      }
   }
   return (ret);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_3_A
// 多角形の面積
Real area(const Polygon& p) {
   Real A = 0;
   for(int i = 0; i < p.size(); ++i) {
      A += cross(p[i], p[(i + 1) % p.size()]);
   }
   return A * 0.5;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_H
// 円と多角形の共通部分の面積
Real area(const Polygon& p, const Circle& c) {
   if(p.size() < 3) return 0.0;
   function<Real(Circle, Point, Point)> cross_area =
       [&](const Circle& c, const Point& a, const Point& b) {
          Point va = c.p - a, vb = c.p - b;
          Real f = cross(va, vb), ret = 0.0;
          if(eq(f, 0.0)) return ret;
          if(max(abs(va), abs(vb)) < c.r + EPS) return f;
          if(distance(Segment(a, b), c.p) > c.r - EPS)
             return c.r * c.r * arg(vb * conj(va));
          auto u = crosspoint(c, Segment(a, b));
          vector<Point> tot{a, u.first, u.second, b};
          for(int i = 0; i + 1 < tot.size(); i++) {
             ret += cross_area(c, tot[i], tot[i + 1]);
          }
          return ret;
       };
   Real A = 0;
   for(int i = 0; i < p.size(); i++) {
      A += cross_area(c, p[i], p[(i + 1) % p.size()]);
   }
   return A;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_4_B
// 凸多角形の直径(最遠頂点対間距離)
Real convex_diameter(const Polygon& p) {
   int N = (int)p.size();
   int is = 0, js = 0;
   for(int i = 1; i < N; i++) {
      if(p[i].imag() > p[is].imag()) is = i;
      if(p[i].imag() < p[js].imag()) js = i;
   }
   Real maxdis = norm(p[is] - p[js]);

   int maxi, maxj, i, j;
   i = maxi = is;
   j = maxj = js;
   do {
      if(cross(p[(i + 1) % N] - p[i], p[(j + 1) % N] - p[j])
         >= 0) {
         j = (j + 1) % N;
      } else {
         i = (i + 1) % N;
      }
      if(norm(p[i] - p[j]) > maxdis) {
         maxdis = norm(p[i] - p[j]);
         maxi = i;
         maxj = j;
      }
   } while(i != is || j != js);
   return sqrt(maxdis);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_5_A
// 最近点対
Real closest_pair(Points ps) {
   if(ps.size() <= 1) throw(0);
   sort(begin(ps), end(ps));

   auto compare_y = [&](const Point& a, const Point& b) {
      return imag(a) < imag(b);
   };
   vector<Point> beet(ps.size());
   const Real INF = 1e18;

   function<Real(int, int)> rec = [&](int left, int right) {
      if(right - left <= 1) return INF;
      int mid = (left + right) >> 1;
      auto x = real(ps[mid]);
      auto ret = min(rec(left, mid), rec(mid, right));
      inplace_merge(begin(ps) + left, begin(ps) + mid,
                    begin(ps) + right, compare_y);
      int ptr = 0;
      for(int i = left; i < right; i++) {
         if(abs(real(ps[i]) - x) >= ret) continue;
         for(int j = 0; j < ptr; j++) {
            auto luz = ps[i] - beet[ptr - j - 1];
            if(imag(luz) >= ret) break;
            ret = min(ret, abs(luz));
         }
         beet[ptr++] = ps[i];
      }
      return ret;
   };
   return rec(0, (int)ps.size());
}
```

input.md

空白を含む文字列の入力

cin >> s は空白で切れる。行をそのまま取るなら getline。

1行だけ読む

>> を一度も使っていないなら、そのまま呼ぶだけでよい。

```
string s;
getline(cin, s);   // 空白を含む1行全体
```

ignore が要るのは >> と混ぜたときだけ。

個数を読んでから行を読む

```
int n;
cin >> n;
cin.ignore(1000, '\n');   // cin >> が残した改行を捨てる
vector<string> v(n);
rep(i, n) getline(cin, v[i]);
```

ignore を忘れると 1本目が空文字列になり、以降が1つずつ ずれる。

ignore が要るのは >> の直後に getline するときだけ。 getline を続けて呼ぶ分には不要。

ignore の形

- cin.ignore() は1文字だけ捨てる。5 のように行末に 空白があると改行に届かず、getline が " " を拾う
- cin.ignore(1000, '\n') は改行を1つ捨てるまで読み飛ばす。 厳密には 1000 の代わりに numeric_limits<streamsize>::max()

cin >> ws との違い

cin >> ws でも改行は飛ぶが 空行も飛ばす。空行が意味を持つ 入力では使わない。ignore は改行を1つだけ捨てるので安全。

行を単語に割る

```
istringstream ss(s);
string w;
while(ss >> w) { /* w を使う */ }
```

個数が分からない整数列も同じ形で取れる。

```
string line;
getline(cin, line);
istringstream ls(line);
ll x;
vll v;
while(ls >> x) v.push_back(x);
```

区切り文字を指定（CSV など）

```
string line;
while(getline(cin, line)) {       // まず行で切る
   istringstream ls(line);
   string f;
   while(getline(ls, f, ',')) { /* f を使う */ }
}
```

getline(cin, s, ',') を直接回すと、行末のフィールドに改行が 入ってしまう。行で切ってから分割するのが安全。

行末の \r

Windows 由来の入力だと行末に \r が残り、長さが1大きくなる。

```
if(!s.empty() && s.back() == '\r') s.pop_back();
```

空白入りの名前 + 数値

```
string line;
getline(cin, line);
size_t q = line.rfind(' ');
string name = line.substr(0, q);
ll v = stoll(line.substr(q + 1));
```

John Smith Jr 42 -> 名前 John Smith Jr、値 42。 前から読みたいなら line.find(' ') を使う。

行数が与えられないとき

```
string s;
while(getline(cin, s)) { /* s を使う */ }
```

空行も s = "" として読まれ、EOF で止まる。

性質

- getline は先頭・末尾の空白も、連続する空白もそのまま残す
- 戻り値はストリームなので while の条件に直接書ける
- cin.tie(0) / ios::sync_with_stdio(false) と併用してよい
- ただし scanf / printf と混ぜない
- 1文字ずつ空白込みで読むなら cin.get(c)

---

int128.hpp

md5: a4c180

```
std::ostream& operator<<(std::ostream& dest, __int128_t value)
{
   std::ostream::sentry s(dest);
   if(s) {
      __uint128_t tmp = value < 0 ? -value : value;
      char buffer[128];
      char* d = std::end(buffer);
      do {
         --d;
         *d = "0123456789"[tmp % 10];
         tmp /= 10;
      } while(tmp != 0);
      if(value < 0) {
         --d;
         *d = '-';
      }
      int len = std::end(buffer) - d;
      if(dest.rdbuf()->sputn(d, len) != len) {
         dest.setstate(std::ios_base::badbit);
      }
   }
   return dest;
}

__int128 parse(string& s) {
   __int128 ret = 0;
   for(int i = 0; i < s.length(); i++)
      if('0' <= s[i] && s[i] <= '9')
         ret = 10 * ret + s[i] - '0';
   return ret;
}
/*
int main() {
  string s = "18782187821878218782187821878218782";
  __int128 x = parse(s);
  x *= 2;
  cout << x << endl;
}
*/
```

---

memory.md

メモリと配置

すべて手元（Ubuntu 24.04 / g++ 13 / x86_64）での実測値。

型ごとのバイト数

型	バイト	1024MB で持てる個数
vector<bool> / bitset	1 bit	約 8×10^9
char / bool	1	約 10^9
short	2	約 5×10^8
int / float	4	約 2.5×10^8
ll / double / pair<int,int> / ポインタ	8	約 1.25×10^8
i128	16	約 6×10^7
set / map の1要素	約 40	—
unordered_set / unordered_map の1要素	約 32	—

reserve 無しの push_back はピークが最大2.5倍。

グローバル生配列は触った分だけ課金される

381 MiB を確保して、先頭だけ書き込んだときの実メモリ。

触った量	グローバル生配列	vector
1 MiB	1.0 MB	381.5 MB
95 MiB	95.4 MB	381.5 MB
381 MiB	381.5 MB	381.5 MB

vector<T> v(n) は構築時に全要素をゼロで埋めるので、その場で 全ページに触る。グローバル配列は BSS に置かれ、OS が書き込んだ ページだけを割り当てる。

大きく取って一部しか使わないならグローバル生配列。サイズが 実行時に決まる、関数内で持つ、push_back で伸ばすなら vector。

vector<vector<>> は1行あたり約38バイト

vector 本体 24 バイト + malloc のヘッダ等。

形	理論値	vv	フラット
1000 × 1000	3.8 MB	3.9 MB	3.9 MB
100000 × 100	38.1 MB	42.0 MB	38.1 MB
2000000 × 4	30.5 MB	106.9 MB	30.5 MB

判断は 行数 × 38 バイト が無視できるかだけ。1行が数要素しか ないときは必ずフラットにして添字を自分で計算する。

deque は空でも約656バイト

	空	1個入り
vector を 100万個	22.9 MB	53.4 MB
deque を 100万個	656.1 MB	656.1 MB

空でも 512 バイトのノードを確保するため。1個なら誤差だが、10^5 個以上並べるなら vector にする。中身が入っていれば メモリはほぼ同じ（10^7 要素で 83.6 対 79.8 MB）。

vector は1bitだが遅い

4×10^8 要素で測ると

	メモリ	書き込み+読み出し1往復
vector<bool>	48 MB	1.17 s
vector<char>	384 MB	0.21 s

1要素ごとにシフトとマスクが入るので 5.6 倍遅い。メモリに余裕 があるなら char。

アクセスの速さは配列サイズではなくパターンで決まる

2×10^8 回アクセスしたときの速度。

配列サイズ	連続	ランダム
256KB	3243 M/s	2257 M/s
4MB	2418 M/s	444 M/s
32MB	2581 M/s	152 M/s
512MB	1243 M/s	40 M/s

連続アクセスは配列サイズにほぼ依存しない。 ランダムは キャッシュに収まるかで決まる。見積もりは

```
連続      10^9 回/秒
ランダム  10^8 回/秒（キャッシュ外）
```

計算量だけでなく「ランダムアクセスが何回か」を数える。

bitDP はフラグを整数に詰める

bool ok[n][1<<n] のように「状態ごとに n 個のフラグ」を持つと、ok[v][bit] が 16 MB 飛びになってキャッシュが効かない。 $n \le 32$ なら int 1個に詰める。

```
int ok[1 << 24];              // ok[bit] の v ビット目がフラグ
ok[bit] |= 1 << v;            // 構築
int m = ok[bit] & ~bit;       // 使うとき
while(m) {
   int v = __builtin_ctz(m);
   m &= m - 1;
}
```

n = 24、2^{24} 状態で測ると

配置	時間	メモリ
ok[v][bit]（16MB飛び）	2.39 s	384 MiB
ok[bit][v]（連続）	2.02 s	384 MiB
int に詰める	0.28 s	64 MiB

メモリ6分の1、速度8.5倍。添字の入れ替えだけでは効かない。 $n \le 64$ なら ll、それ以上なら bitset の行を持つ。

memsize.cpp

md5: dcc941

```cpp
// 型と要素数を入れると MB が出る
//   int 44000000  ->  176.0 MB
// 型名の代わりに1要素のバイト数を直接書いてもよい
//   12 1000000
// set/map/uset/umap は1要素あたりの実測値
// row は vector<vector<>> の1行あたりの上乗せ
// reserve 無しの push_back はピーク2.5倍
// 実行ファイルと標準ライブラリで数MB
// 目安: 256MB = int 6400万個 = ll 3200万個
#include <bits/stdc++.h>
using namespace std;
int main() {
   map<string, double> B = {{"char", 1},   {"bool", 1},
                            {"short", 2},  {"int", 4},
                            {"float", 4},  {"ll", 8},
                            {"double", 8}, {"pair", 8},
                            {"ptr", 8},    {"i128", 16},
                            {"set", 40},   {"map", 40},
                            {"uset", 32},  {"umap", 32},
                            {"row", 30}};
   string t;
   double n;
   while(cin >> t >> n) {
      double b = B.count(t) ? B[t] : atof(t.c_str());
      printf("%s x %.0f = %.1f MB\n", t.c_str(), n,
             b * n / 1e6);
   }
}
```

random.hpp

md5: bf85b5

```cpp
struct RandomNumberGenerator {
   mt19937 mt;

   RandomNumberGenerator()
       : mt(chrono::steady_clock::now()
                .time_since_epoch()
                .count()) {}

   int operator()(int a, int b) {  // [a, b)
      uniform_int_distribution<int> dist(a, b - 1);
      return dist(mt);
   }

   int operator()(int b) {  // [0, b)
      return (*this)(0, b);
   }
};
void solve(int test) {
   RandomNumberGenerator rd;
   cout << rd(0, 1e9) << endl;
   cout << rd(1e5) << endl;
}
```

range_edge_graph.hpp

md5: f3f96c

```cpp
template<typename W> struct range_edge_graph {
   int n;
   struct edge {
      int to;
      W weight;
   };
   vector<vector<edge>> g;

   range_edge_graph(int n) : n(n) {
      g.resize(4 * n);
```

```
    for(int i = 1; i < n; ++i) {
      int cl = i << 1 | 0, cr = i << 1 | 1;
      g[i].push_back({cl, 0});
      g[i].push_back({cr, 0});
      g[cl + 2 * n].push_back({i + 2 * n, 0});
      g[cr + 2 * n].push_back({i + 2 * n, 0});
    }
    for(int i = n; i < 2 * n; ++i)
      g[i].push_back({i + 2 * n, 0});
  }

  // from [l1, r1) to [l2, r2)
  void add_edge(int l1, int r1, int l2, int r2, W w) {
    int idx = g.size();
    for(l1 += n, r1 += n; l1 < r1; l1 >>= 1, r1 >>= 1) {
      if(l1 & 1) g[l1 + 2 * n].push_back({idx, 0}), l1++;
      if(r1 & 1) --r1, g[r1 + 2 * n].push_back({idx, 0});
    }
    vector<edge> node;
    for(l2 += n, r2 += n; l2 < r2; l2 >>= 1, r2 >>= 1) {
      if(l2 & 1) node.push_back({l2++, w});
      if(r2 & 1) node.push_back({--r2, w});
    }
    g.push_back(node);
  }
```

```
  vector<W> dijkstra(int s) {
    s += n;
    vector<W> dist(g.size(), numeric_limits<W>::max());
    dist[s] = 0;
    using P = pair<W, int>;
    priority_queue<P, vector<P>, greater<P>> que;
    que.emplace(0, s);
    while(!que.empty()) {
      P p = que.top();
      que.pop();
      int v = p.second;
      if(dist[v] < p.first) continue;
      for(edge& e : g[v]) {
        if(dist[e.to] > dist[v] + e.weight) {
          dist[e.to] = dist[v] + e.weight;
          que.emplace(dist[e.to], e.to);
        }
      }
    }
    vector<W> res(dist.begin() + n, dist.begin() + 2 * n);
    return res;
  }
};
```